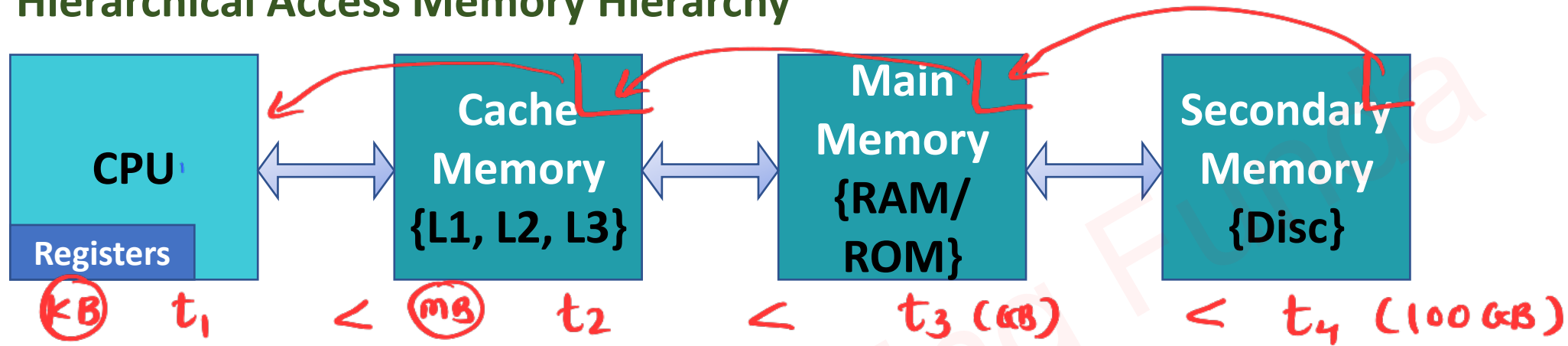
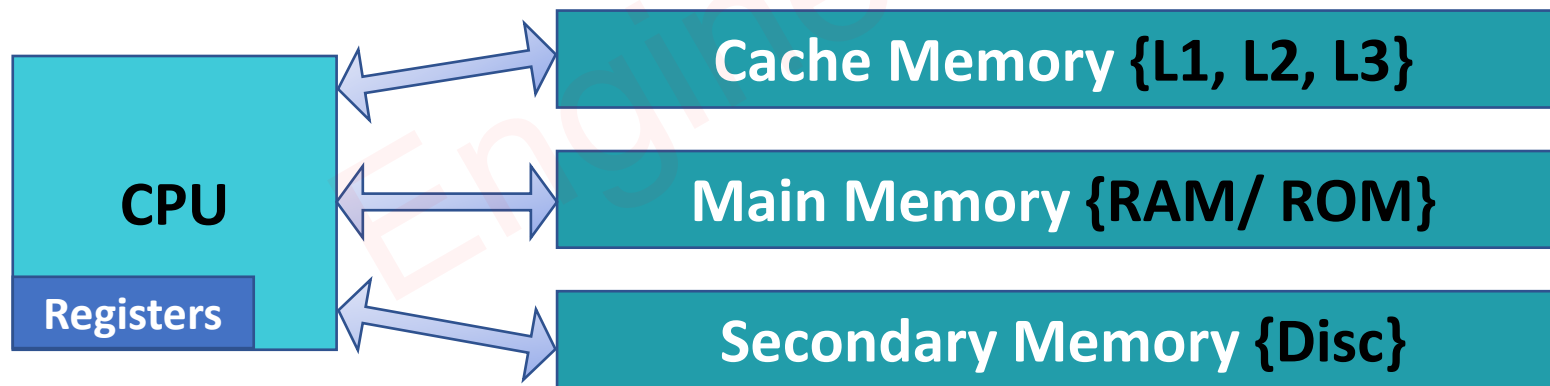


Memory Organization

❖ Hierarchical Access Memory Hierarchy

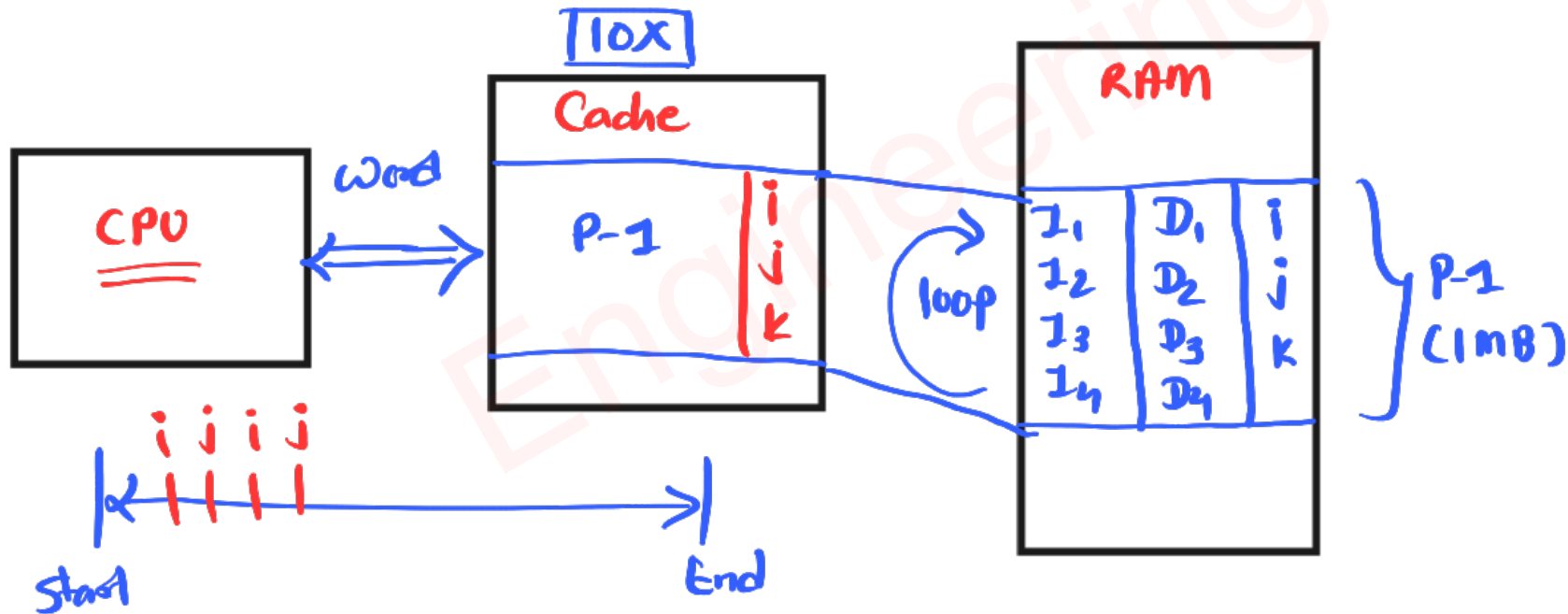
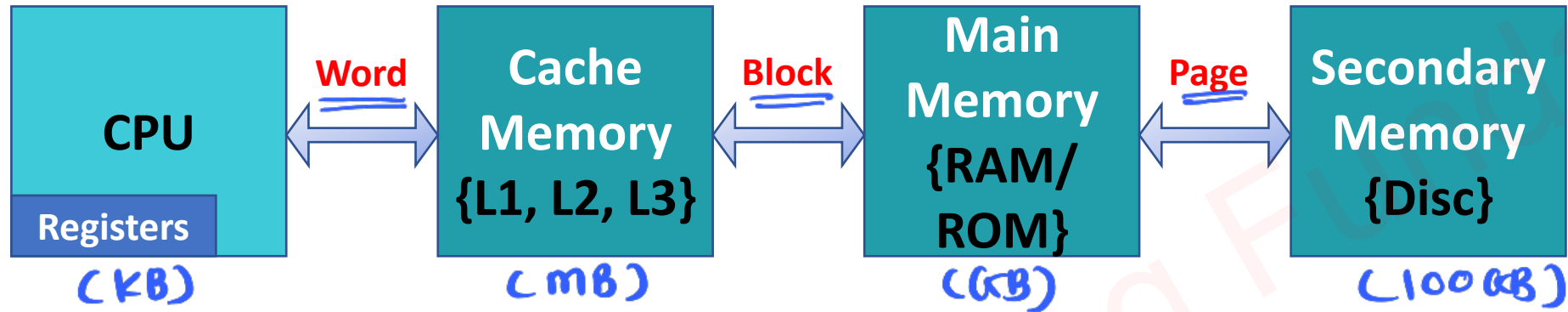


❖ Simultaneous Access Memory Hierarchy



Locality of Reference

❖ Hierarchical Access Memory Hierarchy



❖ Locality of Reference

- ❑ Locality of reference is a principle in computer science and computer architecture that describes the tendency of a computer program to access the same set of memory locations repeatedly over a short period of time.

❖ Spatial Locality

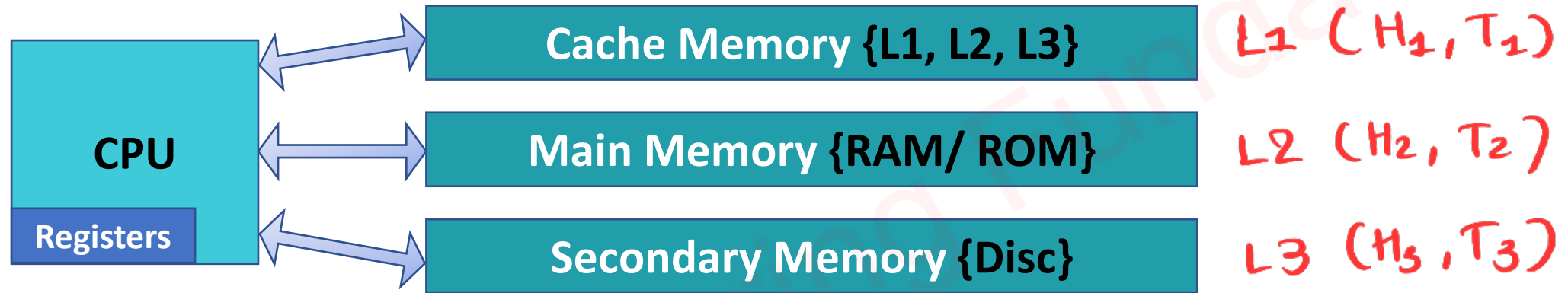
- ❑ During execution of program, if instructions and data are executed in loops then it comes under the category of spatial locality.
- ❑ In spatial locality, addresses of locality is close to each other.

❖ Temporal Locality

- ❑ During execution of program, if variables are used frequently then it comes under the category of temporal locality.

Average Access Time and Access Speed

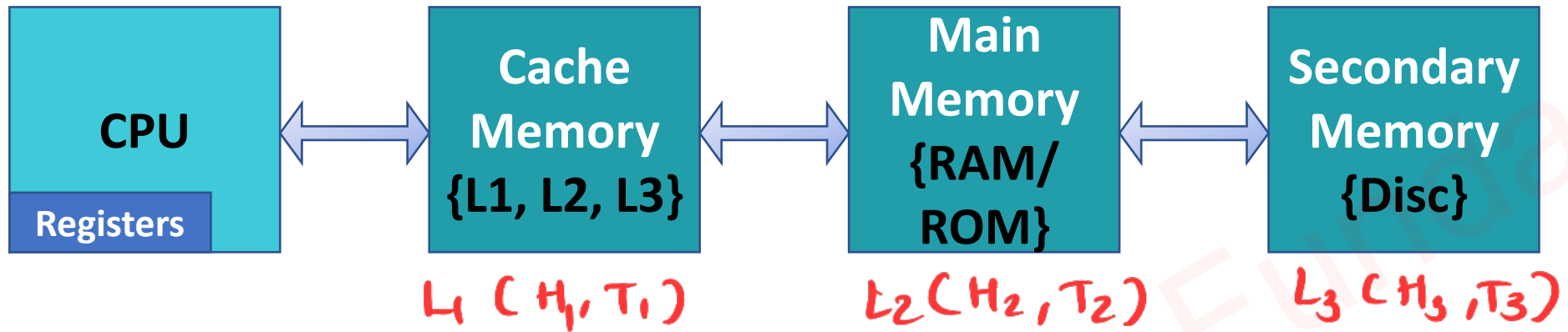
❖ Simultaneous Access Memory Hierarchy Access Time and Access Speed



$$\Rightarrow T_{avg} = H_1 T_1 + (1 - H_1) H_2 T_2 + (1 - H_1) (1 - H_2) H_3 T_3$$

$$\Rightarrow \eta = \frac{1}{T_{avg}}$$

❖ Hierarchical Access Memory Hierarchy Access Time and Access Speed



$$\Rightarrow T_{avg} = H_1 T_1 + (1 - H_1) H_2 (T_1 + T_2) + (1 - H_1)(1 - H_2) H_3 (T_1 + T_2 + T_3)$$

$$\Rightarrow \eta = \frac{1}{T_{avg}}$$

Example of Access Time and Access Speed

- ❖ Two Level simultaneous memory hierarchy is given. L1 is 8 times faster than L2. $T_1 = 20\text{ns}$, $T_1 = T_{\text{avg}} - 40\text{ns}$, find H_1 .
- ❖ Also find H_1 , if Hierarchical Access Memory Hierarchy is given.

$$\Rightarrow T_1 = 20 \text{ nsec}$$

$$T_2 = 8T_1 = 160 \text{ nsec}$$

$$T_{\text{avg}} = 60 \text{ nsec}$$

$$\Rightarrow T_{\text{avg}} = H_1 T_1 + (1 - H_1) T_2$$

$$\Rightarrow 60 = 20 H_1 + (1 - H_1) 160$$

$$\Rightarrow 3 = H_1 + (1 - H_1) 8$$

$$\Rightarrow 7 H_1 = 5$$

$$\Rightarrow \boxed{H_1 = 5/7}$$

$$\Rightarrow T_{avg} = H_1 T_1 + (1-H_1) H_2 (T_1 + T_2)$$

$$\Rightarrow 60 = H_1 (20) + (1-H_1) (180)$$

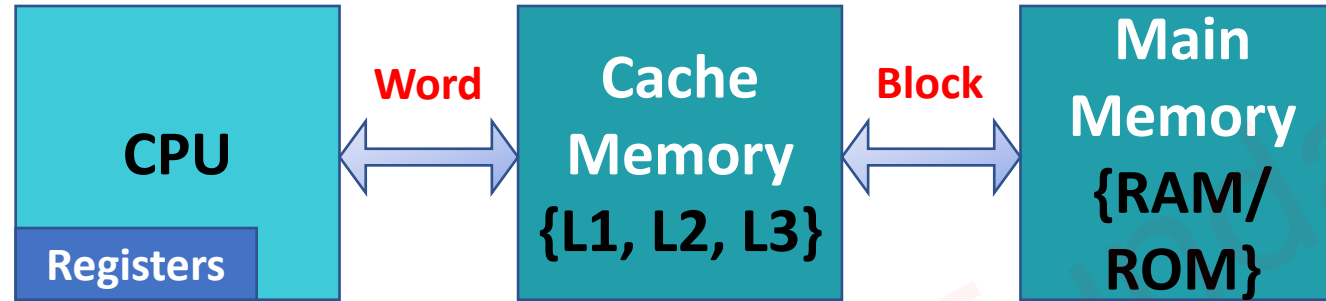
$$\Rightarrow 3 = H_1 + (1-H_1) 9$$

$$\Rightarrow 8H_1 = 6$$

$$\Rightarrow \boxed{H_1 = 3/4}$$

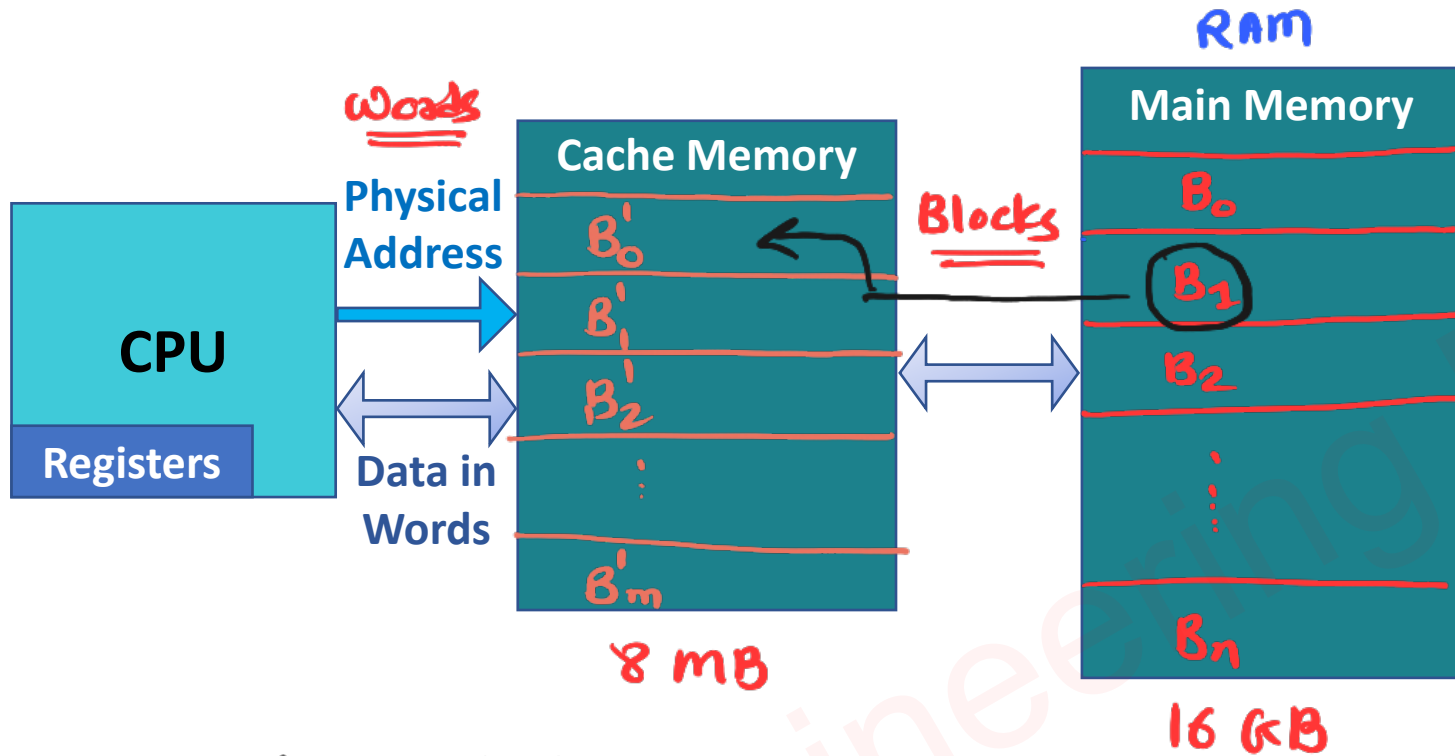
Engineering Funda

Cache Memory



- ❖ Cache Memory Organization
- ❖ Mapping Techniques of Cache Memory
- ❖ Replacement Methods in Cache Memory
- ❖ Data Update with Cache Memory
- ❖ Multi Level Cache Memory

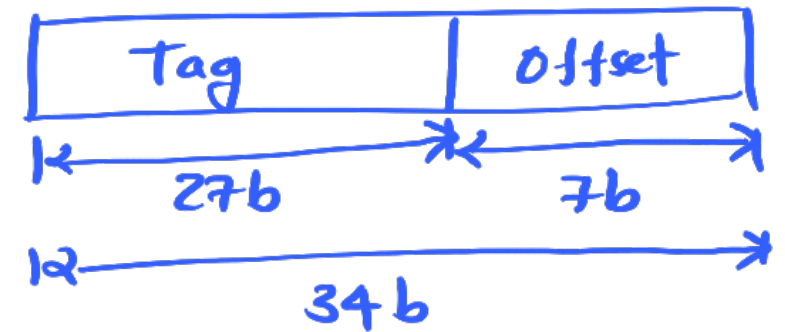
Cache Memory Organization



⇒ Size of block = 128 byte
 $= 2^7$ byte

⇒ Number of blocks in mm
 $= \frac{2^{34}}{2^7} = 2^{27}$

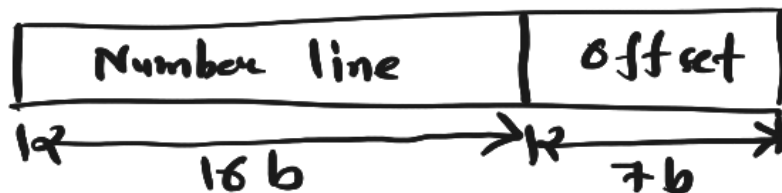
⇒ physical Address



⇒ Numbers of lines in Cache

$$= \frac{2^{23}}{2^7} = 2^{16}$$

⇒ Cache Index



Cache Memory

V/I	Tag	Data
1 bit	27 b tag	B0' = B ₁
		B1'
		...
		Bm'

$$\frac{B_0}{cm} \leftarrow \frac{B_1}{mm}$$

→ physical size of cache

$$= m \times (1 + \text{No of bits of tag} + \text{Size of 1 line})$$

$$= 2^{16} \times (1 + 27 + 128 \times 8)$$

$$= 1052 \times 2^{16}$$

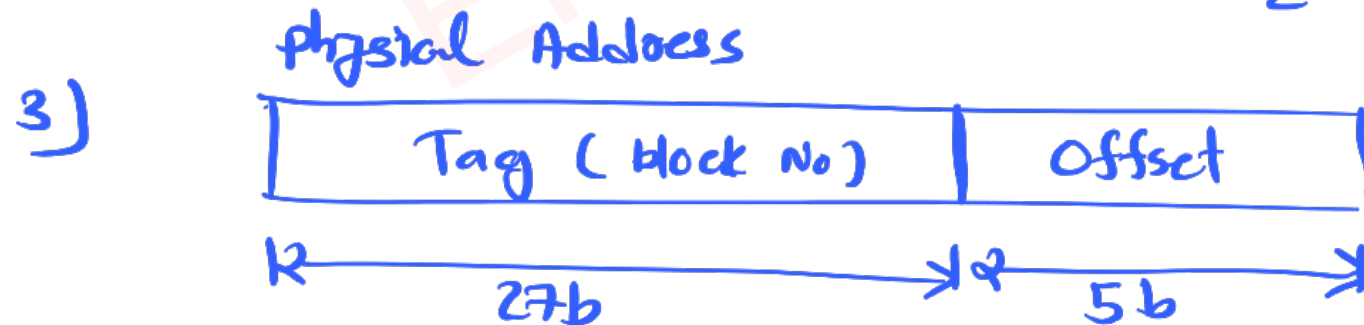
Example of Cache Memory Organization

❖ Size of Cache memory is 32KB. Size of Block is 32B. If CPU generates 32bits of physical address then find,

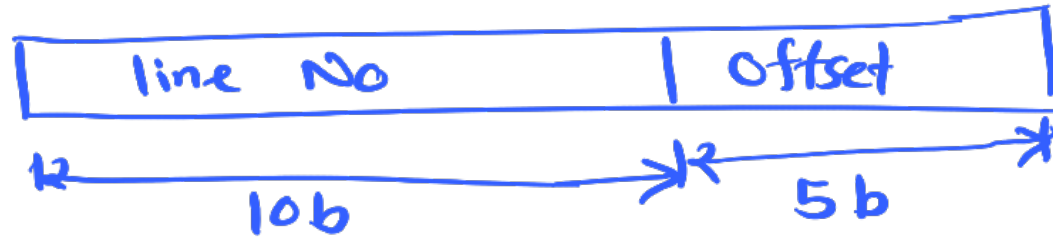
1. Numbers of Lines in Cache
2. Blocks in Main Memory
3. Format of Physical address & Cache Index
4. Physical size of cache memory

1) Numbers of lines in Cache = $\frac{32 \times 2^{10}}{32} = 2^{10}$

2) Blocks in main memory = $\frac{2^{32}}{2^5} = 2^{27}$



Cache Index



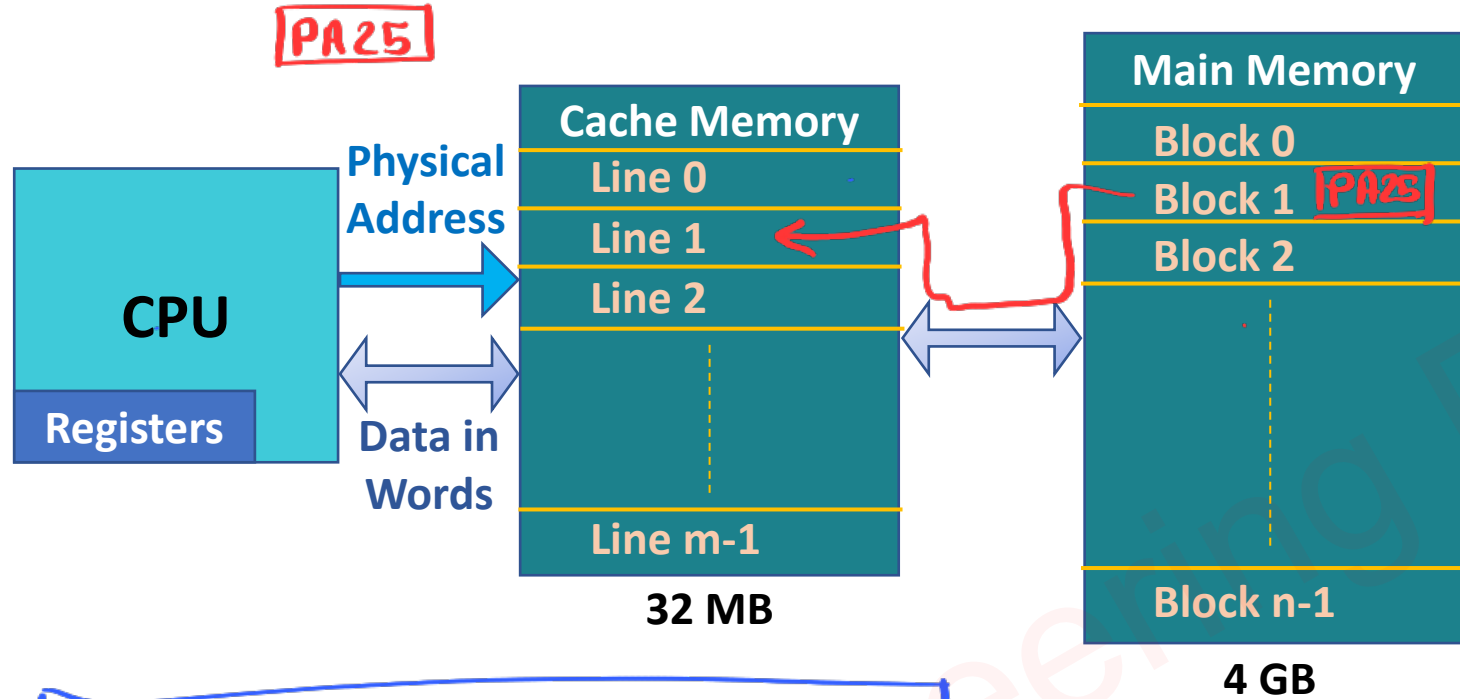
4) physical size of cache memory

$$= m \times (1 + \text{Tag bits} + \text{Size of line in bits})$$

$$= 2^{10} \times (1 + 27 + 32 \times 8)$$

$$= 284 \times 2^{16}$$

Direct Address Mapping in Cache Memory



❖ If size of Block is 16KB = 2^{14} Bytes.

❖ So, Number of Blocks = $\frac{2^{32}}{2^{14}} = 2^{18}$

❖ So, Number of Lines = $\frac{2^{25}}{2^{14}} = 2^{11}$

❖ Physical Address

Tag (Block #) {18b}

Offset {14b}

❖ Cache Index

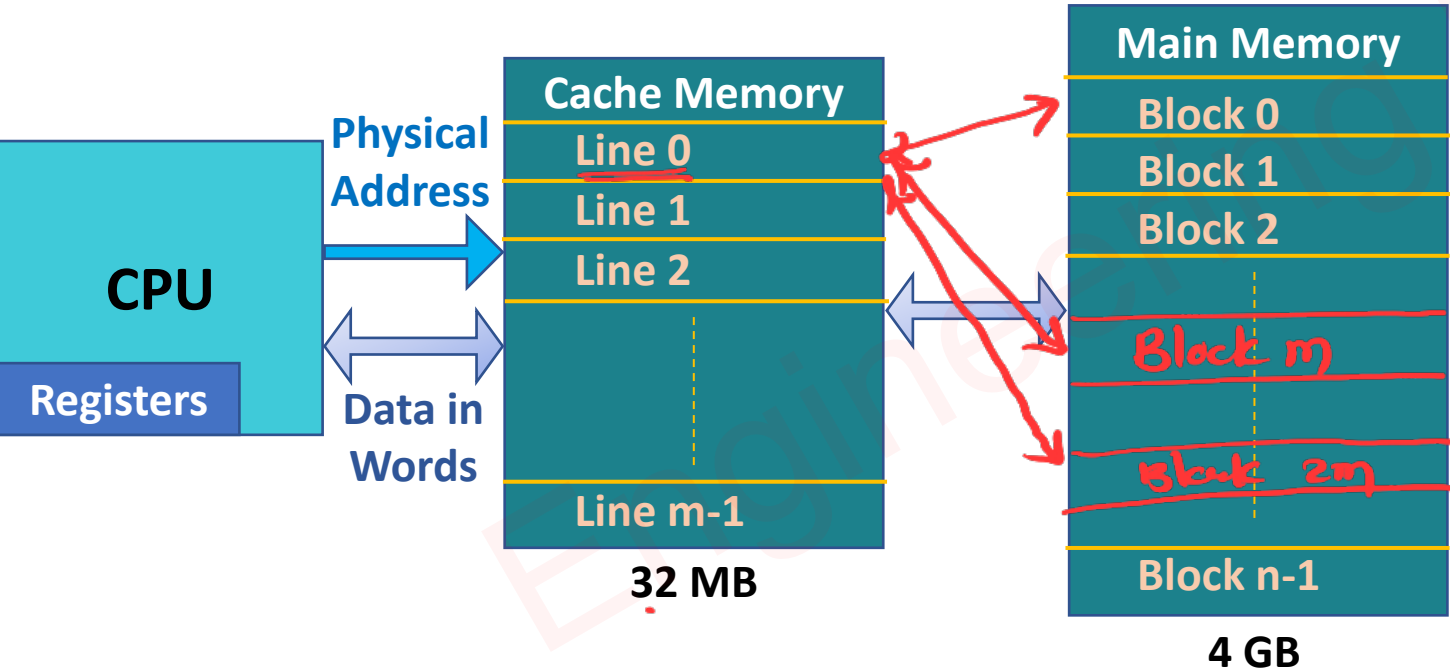
Line Number {11b}

Offset {14b}

$$\text{Line \#} = \text{Tag} \bmod m$$

$$\begin{array}{lcl} \rightarrow 17 \bmod 5 = 2 & | & \underline{11000} \bmod \underline{100} = \underline{00} \\ 17 \bmod 4 = 1 & | & \underline{11001} \bmod \underline{100} = \underline{01} \\ 15 \bmod 3 = 0 & | & \underline{11010} \bmod \underline{100} = \underline{10} \end{array}$$

L0	L1	L2	L3		Lm-1
B0	B1	B2	B3		Bm-1
Bm	Bm+1	Bm+2	Bm+3		B2m-1
B2m	B2m+1	B2m+2	B2m+3		B3m-1
...	...	...	...		...



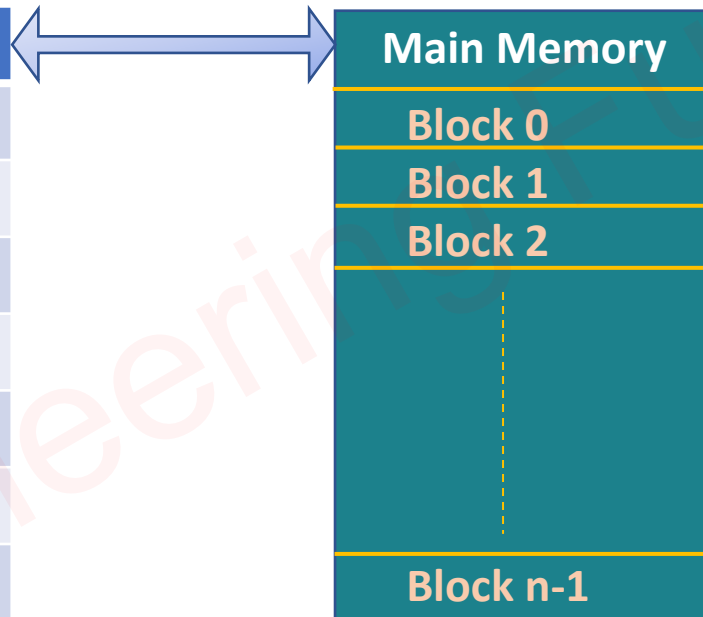
❖ Physical Address

Tag (Block #) {18b}	Offset {14b}
---------------------	--------------

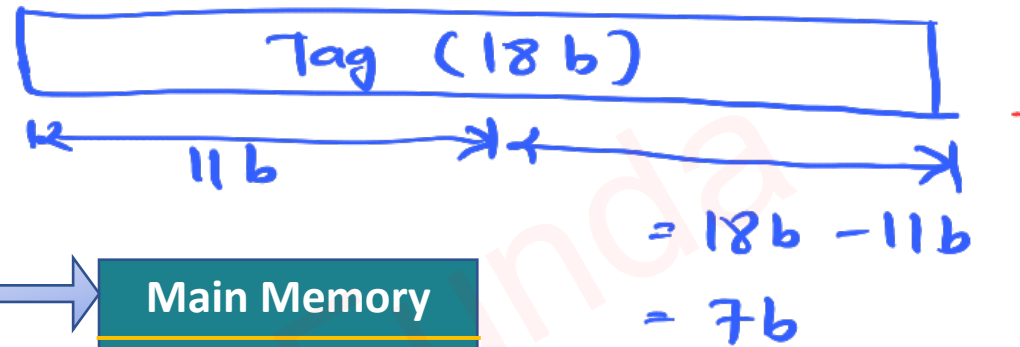
❖ Cache Index

Line Number {11b}	Offset {14b}
-------------------	--------------

V/I	Tag (7b)	Data
		L0
		L1
		L2
		...
		...
		...
		Lm-1



$$\text{Line \#} = \text{Block \#} \bmod m$$



$$\Rightarrow \text{Size of Cache} = m \times (1 \text{ bit (V/I)} + (7 \text{ bit (tag)} + \text{data in line (bits)}))$$

❖ Limitations of direct address mapping

Address	Tag {18b tag}	Data with respect to block	
A1 -	0000 000 000 0000 0000	B0	LO
A2 -	<u>0000 010</u> 000 0000 0000	B2m	LO
A3	0000 100 000 0000 0000	B4m	LO
A4	0000 001 000 0000 0000	Bm	LO
A5	0000 011 000 0000 0000	B3m	LO

❖ Thrashing : Too much swapping of data

Example of Direct Address Mapping in Cache Memory

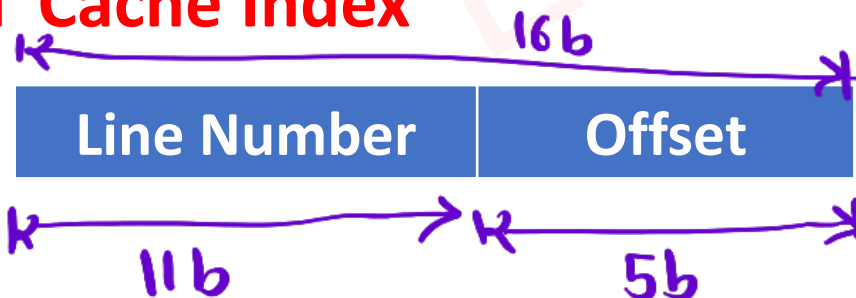
❖ Size of Cache memory is 64KB, Size of Block is 32B, Here cache memory is mapped with direct address mapping, CPU is generating 32bits of physical address, here cache controller has one bit for validity and one bit for modification then find:

1. Tag bits (16 bits)
2. Size of tag memory in given cache memory
3. Size of cache memory

Physical Address



Cache Index

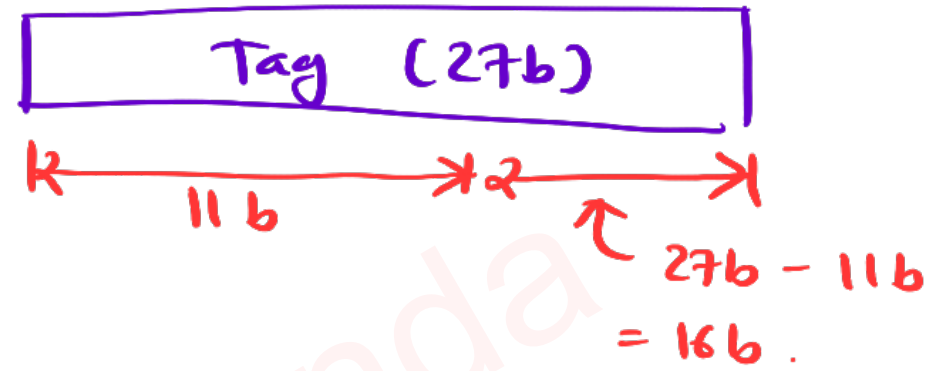


$$\text{Line \#} = \underline{\text{Block \#}} \bmod \underline{m}$$

1b	1b	16b	
V/I	M	Tag	Data
			L0
			L1
			L2
			...
			...
			...
			Lm-1

Cache Controller Info

$$\begin{aligned}
 \Rightarrow \text{Tag memory} &= \text{tag size} \times m \\
 &= 16 \times 2^{18} \\
 &= 32 \text{ K bits.}
 \end{aligned}$$



$$\begin{aligned}
 \Rightarrow \text{Cache Controller} &= 2^{18} \times (1 + 1 + 16) \\
 &= 18 \times 2^{18} \\
 &= 36 \text{ Kb} \\
 &= 4.5 \text{ KB}
 \end{aligned}$$

$$\Rightarrow \text{Size Cache memory} = m \times (\underbrace{1}_{\text{line \#}} + \underbrace{1}_{\text{V/I bit}} + \underbrace{16}_{\text{m bit}} + \underbrace{32 \times 8}_{\text{Tag}})$$

line Size in terms bits.

$$= 2^{11} \times (1 + 1 + 16 + 32 \times 8)$$

$$= 2048 \times 2^{11}$$

$$= 548 \times 2^{10} \text{ bits}$$

$$= 68.5 \times 2^{10} \text{ Byte}$$

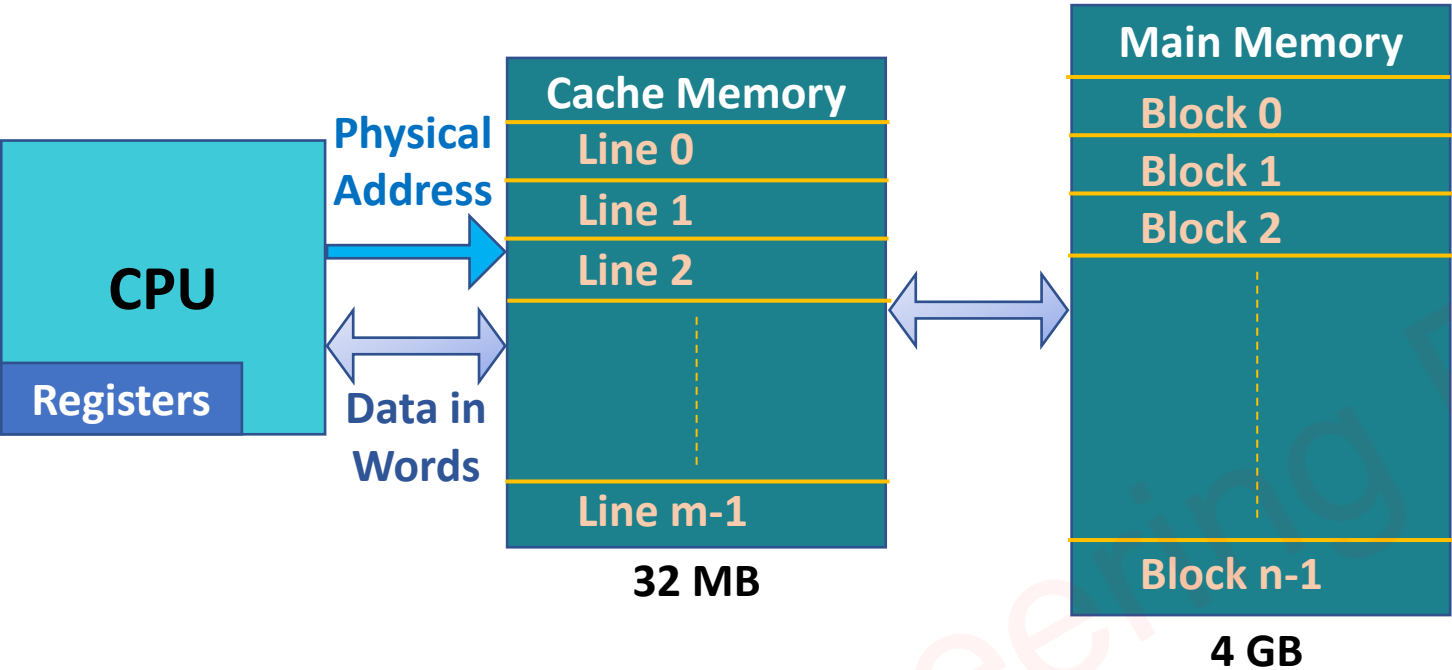
$$= \underline{68.5 \text{ KB}}$$

$$\Rightarrow \text{Size Cache memory} = \text{Cache Controller} + \text{Cache data}$$

$$= 4.5 \text{ KB} + 64 \text{ KB}$$

$$= \underline{68.5 \text{ KB}}$$

Full Associative Address Mapping in Cache Memory



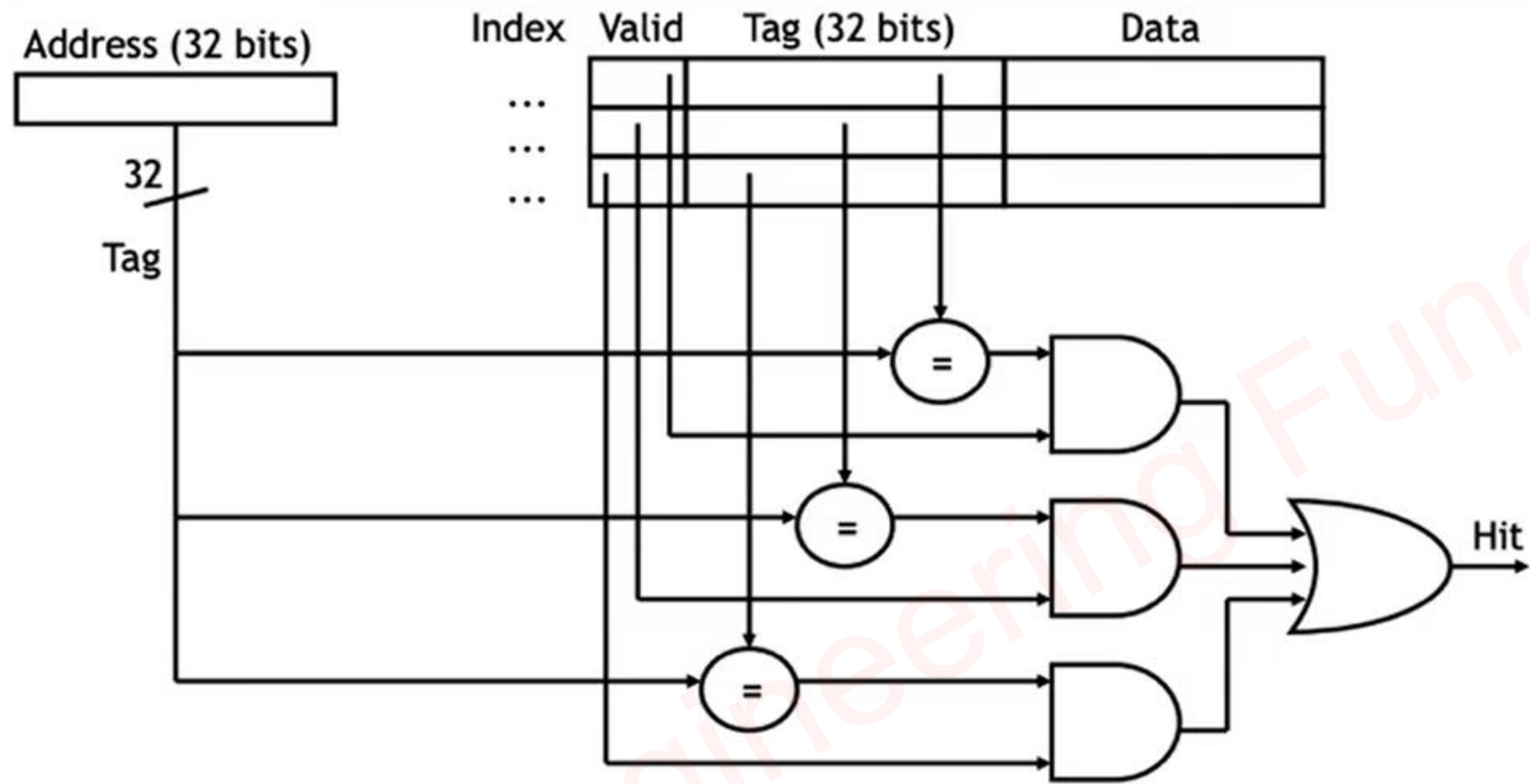
- ❖ If size of Block is 16KB = 2^{14} Bytes.
- ❖ So, Number of Blocks = $\frac{2^{32}}{2^{14}} = 2^{18}$
- ❖ So, Number of Lines = $\frac{2^{25}}{2^{14}} = 2^{11}$
- ❖ Physical Address

Tag (Block #) {18b}	Offset {14b}
---------------------	--------------

- ❖ Cache Index

Line Number {11b}	Offset {14b}
-------------------	--------------

V/I	Tag (18b)	Data
		L0
		L1
		L2
		...
		Lm-1



⇒ physical ckt

Example of Full Associative Address Mapping in Cache Memory

❖ Size of Cache memory is 64KB, Size of Block is 32B, Here cache memory is mapped with full associative address mapping, CPU is generating 32bits of physical address, here cache controller has one bit for validity and one bit for modification and two replacement bits then find:

- 1. Tag bits = 27b
- 2. Size of cache memory controller
- 3. Size of cache memory

$\rightarrow \# \text{Blocks} = 2^{27}$
 $\rightarrow \# \text{lines} = 2^{11}$

Physical Address



Cache Index



16b

1b	1b	2b	27b	
V/I	M	Rep	Tag	Data
				L0
				L1
				L2
				Cache data
				...
				...
				Lm-1

Cache
Controller

Cache data

Cache memory.

Size of Cache

$$\text{Controller} = (1 + 1 + 2 + 27) \times 2^n$$

$$= 31 \times 2^n \text{ bits}$$

$$= 62 \times 2^{10} \text{ bits}$$

$$= 62 \text{ Kb}$$

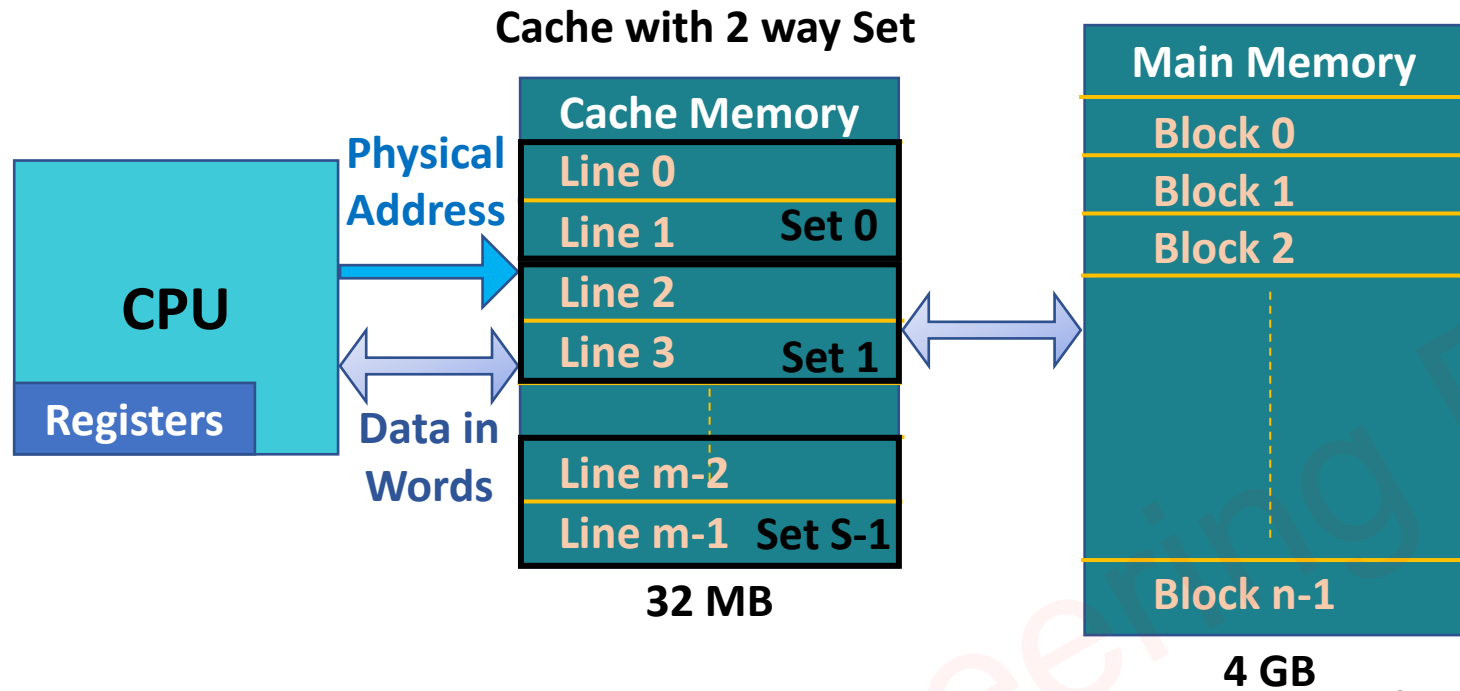
$$= 7.75 \text{ KB}$$

$$\rightarrow \text{Size of Cache memory} = \text{Cache Controller} + \text{Cache data}$$

$$= 7.75 \text{ KB} + 64 \text{ KB}$$

$$= 71.75 \text{ KB}$$

Set Associative Address Mapping in Cache Memory



❖ If size of Block is 1KB = 2^{14} Bytes.

❖ So, Number of Blocks $n = \frac{2^{32}}{2^{14}} = 2^{18}$

❖ So, Number of Lines $m = \frac{2^{25}}{2^{14}} = 2^{11}$

❖ Physical Address

Block Number {18b}	Offset {14b}
--------------------	--------------

❖ Cache Index

Line Number {11b}	Offset {14b}
-------------------	--------------

→ Number of set = $\frac{\text{Number of lines}}{k} = \frac{2^{11}}{2} = 2^{10}$

→ P.A. →

Block #	Offset
18b	14b

tag	Set Index	Offset
8b	10b	14b

→ Set Index = $\text{Block \#} \bmod S$

Cache Controller Cache Data

V/I	Tag	Data
		B_{35} L0
		B_{15} L1 Set 0
		L2
		L3 Set 1
		...
		Lm-2
		Lm-1 Set S-1

- ❖ 1 way set associative is direct address mapping
- ❖ m way set associative is Full associative address mapping.

$B_0, B_s, B_{2s}, \dots$

↓
m Comparators

$B_1, B_{s+1}, B_{2s+1}, \dots$

Address	Tag {18b tag}	Data with respect to block
A1	<u>0000 0000 00 0000 0000</u>	<u>B₀</u>
A2	0000 0010 <u>00 0000 0000</u>	B _{2s}
A3	0000 0100 <u>00 0000 0000</u>	B _{4s}
A4	0000 0001 <u>00 0000 0000</u>	B _{1s}
A5	0000 0011 <u>00 0000 0000</u>	B _{3s}



Example of Set Associative Address Mapping in Cache Memory

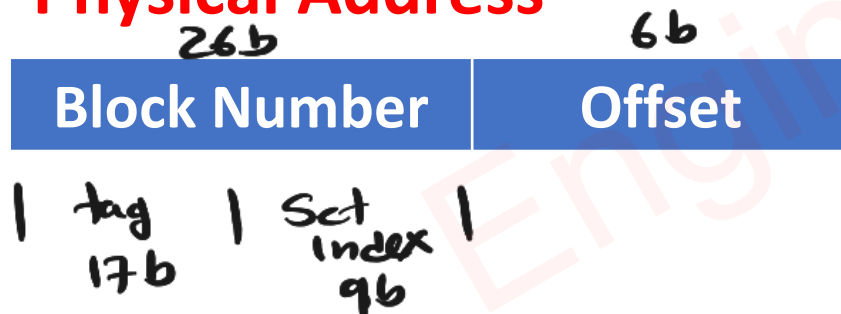
❖ Size of Cache memory is 128B , Size of Block is 64B , Here cache memory is mapped with 4 way set associative address mapping, CPU is generating 32bits of physical address, here cache controller has one bit for validity and one bit for modification and two replacement bits then find:

1. Tag bits = 17b
2. Size of cache memory controller
3. Size of cache memory

$$\rightarrow \text{Number of lines} = \frac{2^{17}}{2^6} = 2^{11}$$

$$\rightarrow \text{Number of Sets} = \frac{\text{lines \#}}{K} = \frac{2^{11}}{4} = 2^9$$

❑ Physical Address



❑ Cache Index



1b	1b	2b	17b	
V/I	M	Rep	Tag	Data
				L0
				L1
				L2
				L3
				L4
				L5
				L6
				L7
				...
				Lm-4
				Lm-3
				Lm-2
				Lm-1

Cache memory Controller

Set 0

Set 1

Set s-1

→ Cache Memory Controller

$$= (1 + 1 + 2 + 17) \times 2^{10} \text{ b}$$

$$= 21 \times 2^{10}$$

$$= 42 \times 2^{10}$$

$$= 42 \text{ Kb}$$

$$= 5.25 \text{ KB}$$

→ Total Size of Cache memory

$$= \text{Cache memory Controller} + \text{Cache data}$$

$$= 5.25 \text{ KB} + 128 \text{ KB}$$

$$= 133.25 \text{ KB}$$

Replacement Strategies in Cache Memory

V/I	Tag	Data
1	✓	L0 = B_{2m}
1	✓	L1 = B_{m+1}
		L2
		...
		...
		...
		Lm-1

Direct Address Mapping

V/I	Tag	Data
1	✓	L0 = B_0
1	✓	L1 = B_m
1	✓	L2 = B_1
		...
		...
		...
		Lm-1

Fully Associative Address Mapping

❖ Physical Address sequence – $B_0, B_m, B_1, B_{2m}, B_{m+1}, B_{2m+1}$

Replacement Strategies in Cache Memory

V/I	Tag	Data
1	✓	L0 = B ₃₅
1	✓	L1 = B ₄₅
1	✓	L2 = B ₂₅
1	✓	L3 = B ₄₅ Set 0
		L4
		L5
		L6
		L7 Set 1
		...
		Lm-4
		Lm-3
		Lm-2
		Lm-1 Set s-1

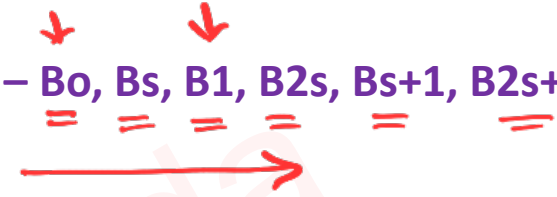
❖ Physical Address sequence – B₀, B_s, B_{2s}, B_s, B_{6s}, B_{2s}, B_{3s}, B_{4s}

Set Associative Address Mapping

FIFO Replacement in Cache Memory

V/I	Tag	Data
1	✓	L0 = B ₂₅
1	✓	L1 = B ₅ Set 0
1	✓	L2 = B ₂₅₊₁
1	✓	L3 = B ₅ Set 1
		...
		Lm-2
		Lm-1 Set S-1

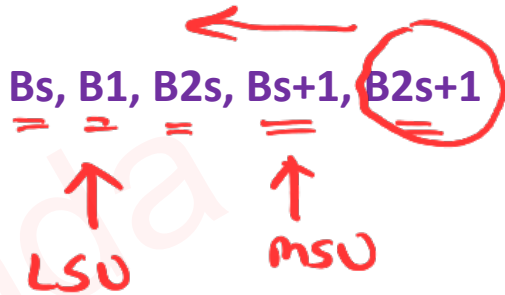
❖ Physical Address sequence – B₀, B_s, B₁, B_{2s}, B_{s+1}, B_{2s+1}



LRU Replacement in Cache Memory

V/I	Tag	Data
1	✓	L0 = B_{2s}
1	✓	L1 = B_s Set 0
1	✓	L2 = B_{2s+1}
1	✓	L3 = B_{s+1} Set 1
		...
		Lm-2
		Lm-1 Set S-1

❖ Physical Address sequence – $B_0, B_s, B_1, B_{2s}, B_{s+1}, B_{2s+1}$



Optimal Replacement in Cache Memory

V/I	Tag	Data
1	✓	L0 = B_2
1	✓	L1 = B_5 Set 0
1	✓	L2 = B_4
1	✓	L3 = B_1 Set 1
		...
		Lm-2
		Lm-1 Set S-1

❖ Physical Address sequence – $B_0, B_s, B_1, B_{2s}, B_{s+1}, B_s$

Engineering Funda

Example on replacement strategies in Cache Memory

- ❖ 4 block cache is given {initially empty}. Order of main memory block references generated by CPU is
4, 5, 7, 12, 4, 5, 13, 4, 5, 7
- ❖ Calculate the hit ratio if
 1. With DAM
 2. 2 way set associative with LRU
 3. 2 way set associative with FIFO

4, 5, 7, 12, 4, 5, 13, 4, 5, 7

V/I	Tag	Data
		L0 = 4
		L1 = 5
		L2
		L3 = 7

DAM

$$\begin{aligned} \text{hit ratio} &= \frac{3}{10} \\ &= 0.3 \\ &= 30\% \end{aligned}$$

V/I	Tag	Data
		L0 = 4
		L1 = 12 Set 0
		L2 = 13
		L3 = 7 Set 1

FIFO

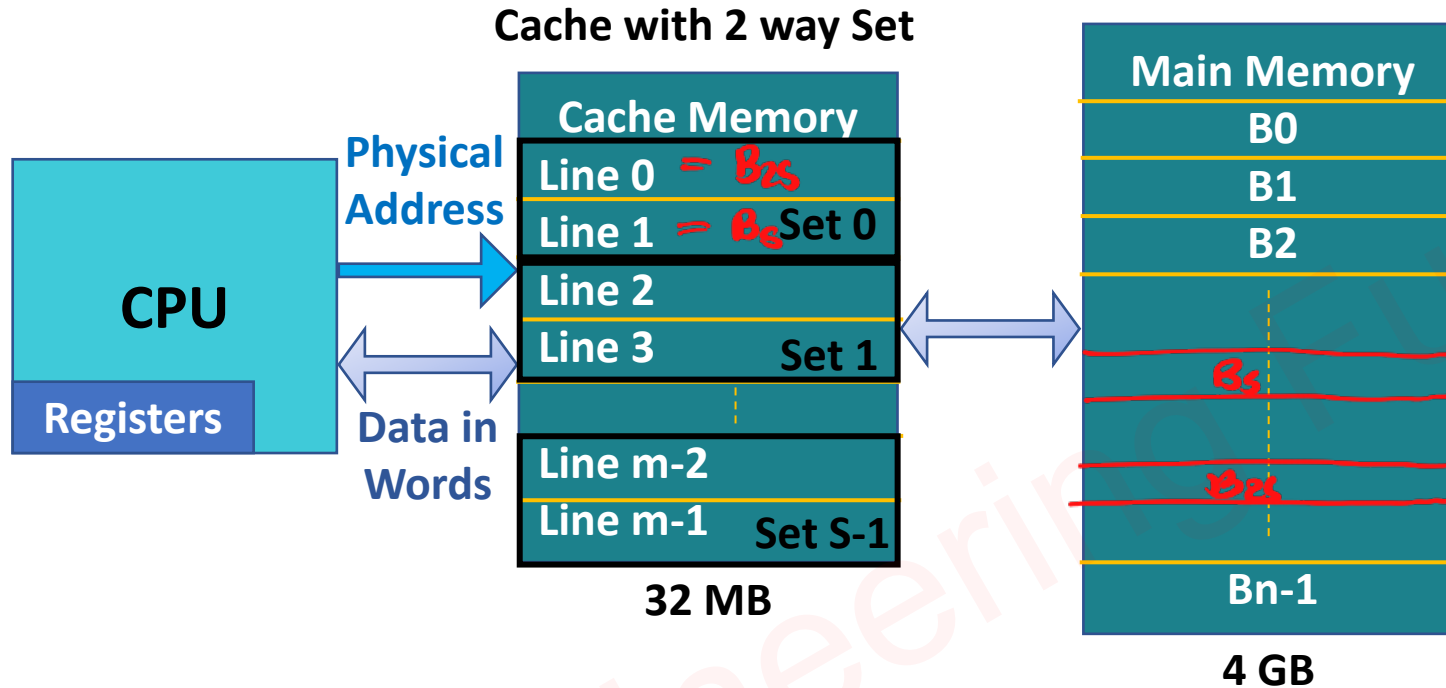
$$\begin{aligned} \text{hit ratio} &= \frac{3}{10} \\ &= 0.3 \\ &= 30\% \end{aligned}$$

V/I	Tag	Data
		L0 = 4
		L1 = 12 Set 0
		L2 = 5
		L3 = 7 Set 1

LRU

$$\begin{aligned} \text{hit ratio} &= \frac{4}{10} \\ &= 0.4 \\ &= 40\% \end{aligned}$$

Updation Techniques in Cache Memory

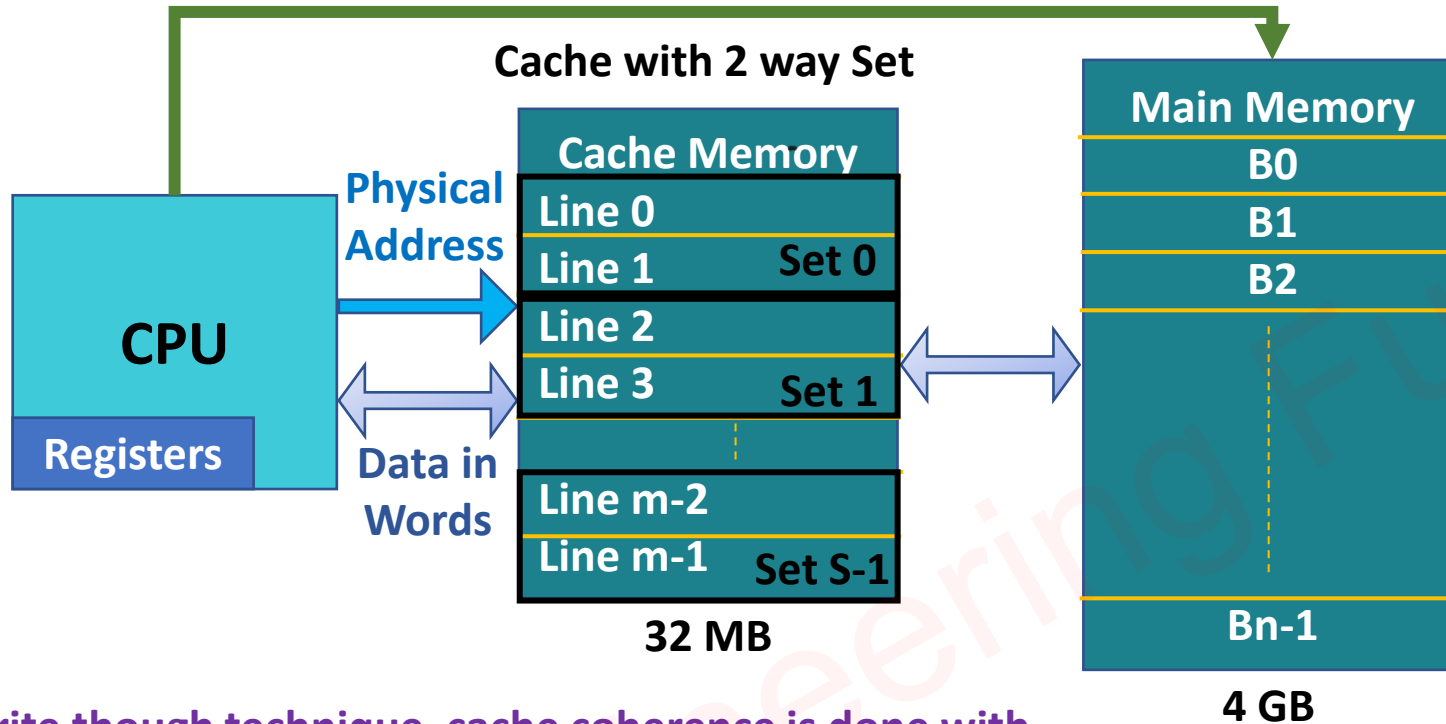


❖ Physical Address sequence – B₀, B_s, B_{2s}, B₀

W R R R

❖ Cache Coherence – It gives idea about modification in cache memory data should also modify the data in main memory.

Write Through Updation Technique in Cache Memory



$$T_{avg} = P_0 T_{avg0} + P_{\infty} T_{avg\infty}$$

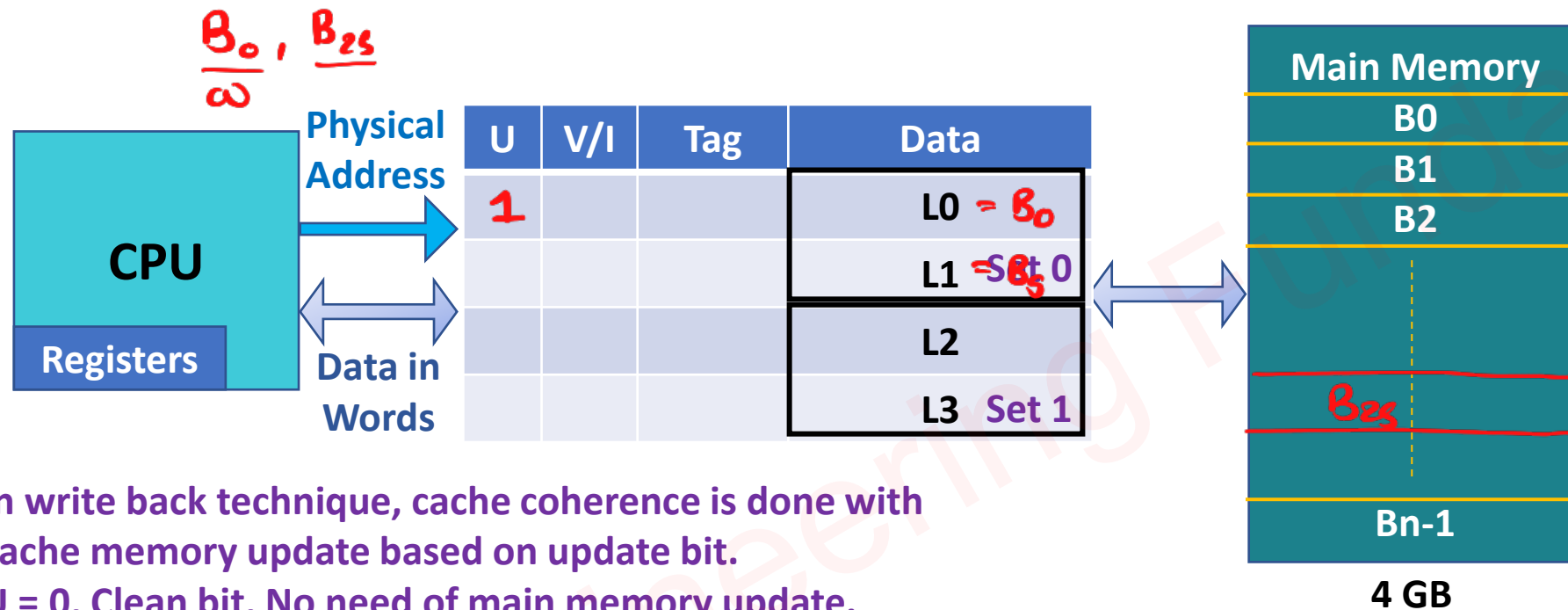
❖ In write through technique, cache coherence is done with simultaneous update in cache memory and main memory.

❖ $T_w = \text{Max (CMw, MMw)}$

$$\Rightarrow T_{avg0} = \frac{H_0 T_{c0}}{\text{Hit}} + \frac{(1 - H_0) (T_{alloc} + T_{mc} + T_{cr})}{\text{Miss}}$$

$$\Rightarrow T_{avg\infty} = \frac{H_{\infty} T_{w}}{\text{Hit}} + \frac{(1 - H_{\infty}) (T_{alloc} + T_{mc} + T_w)}{\text{Miss}}$$

Write Back Updation Technique in Cache Memory



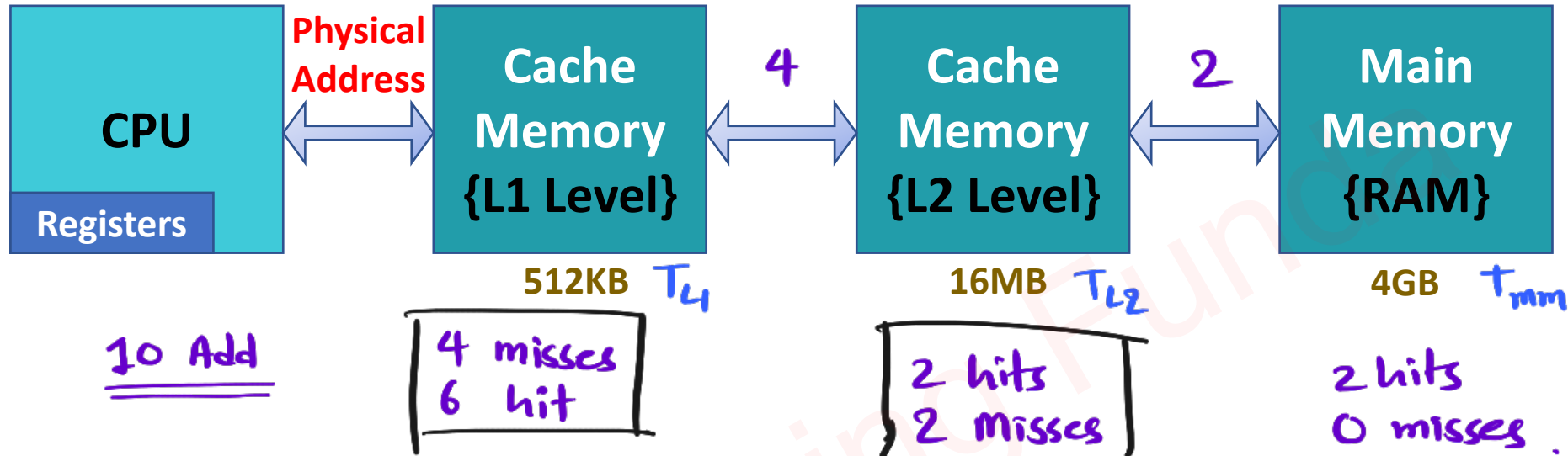
$$T_{avg} = P_0 T_{avg0} + P_\omega T_{avg\omega}$$

- ❖ In write back technique, cache coherence is done with cache memory update based on update bit.
- ❖ U = 0, Clean bit, No need of main memory update.
- ❖ U = 1, Dirty bit, Need of main memory update

$$T_{avg0} = \frac{H_0 T_{c0}}{\text{Hit}} + \frac{(1-H_0)}{\text{Miss}} [P_d (T_{wb} + T_{alloc} + T_{mc} + T_{ca}) + P_c (T_{alloc} + T_{mc} + T_{ca})]$$

$$T_{avg\omega} = \frac{H_\omega T_{c\omega}}{\text{Hit}} + \frac{(1-H_\omega)}{\text{Miss}} [P_d (T_{wb} + T_{alloc} + T_{mc} + T_{c\omega}) + P_c (T_{alloc} + T_{mc} + T_{c\omega})]$$

Multi Level Cache Memory



❖ Miss Penalty : Time taken to service a miss.

❖ Global Miss Rate {GMR} :

$$GMR_{L1} = \frac{4}{10}, \quad GMR_{L2} = \frac{2}{10}, \quad GMR_{mm} = \frac{0}{10}$$

❖ Local Miss Rate {LMR} :

$$LMR_{L1} = \frac{4}{10}, \quad LMR_{L2} = \frac{2}{4}, \quad LMR_{mm} = \frac{0}{2}$$

❖ Average Access Time:

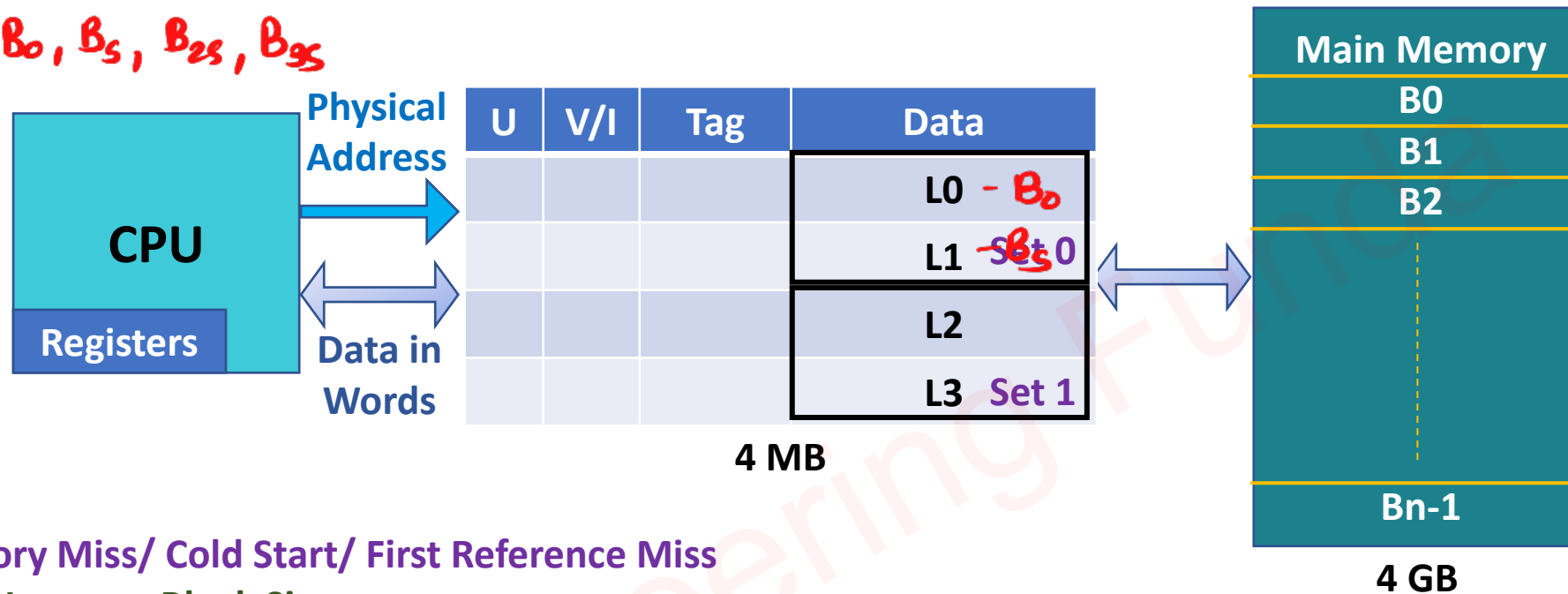
$$\Rightarrow T_{avg} = H_{L1} \times T_{L1} + (1 - H_{L1}) \times \text{Miss penalty of } L1$$

$$\Rightarrow \text{Miss penalty of } L1 = H_{L2} \times T_{L2} + (1 - H_{L2}) \times \text{Miss penalty of } L2$$

$$\Rightarrow \text{Miss penalty of } L2 = T_{mm}$$

Types of misses in Cache Memory

B_0, B_5, B_{25}, B_{95}



❖ Compulsory Miss/ Cold Start/ First Reference Miss

❑ Solution: Increase Block Size

❖ Capacity Miss

❑ Solution: Increase Cache Size

❖ Conflict Miss

❑ Solution: Increase K way in Cache Memory

Examples on mapping of Cache Memory

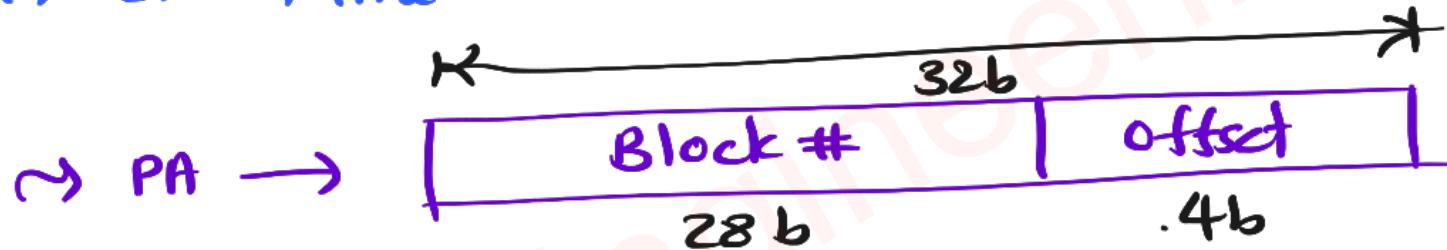
- ❖ A certain processor uses a fully associative cache of size 16KB, The cache block size is 16 Bytes. Assume that the main memory is byte addressable and uses a 32 bit address. How many bits are required for the Tag and the Index fields respectively in the addresses generated by the processor?

0b

- A. 24 bits and 0 bits
- B. 28 bits and 4 bits ✗
- C. 24 bits and 4 bits ✗
- ✓ D. 28 bits and 0 bits

⇒ Cache memory Size = 16 KB = 2^{14} Bytes

⇒ Block/line size = 16 B = 2^4 Bytes



⇒ Tag = 28 b

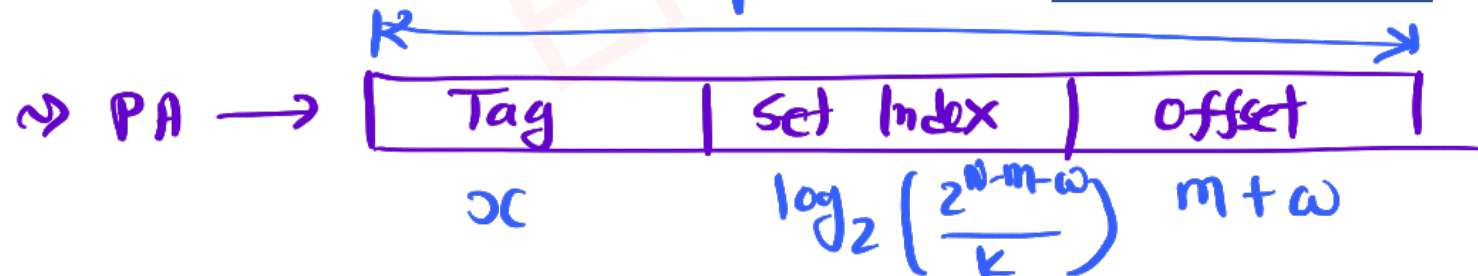
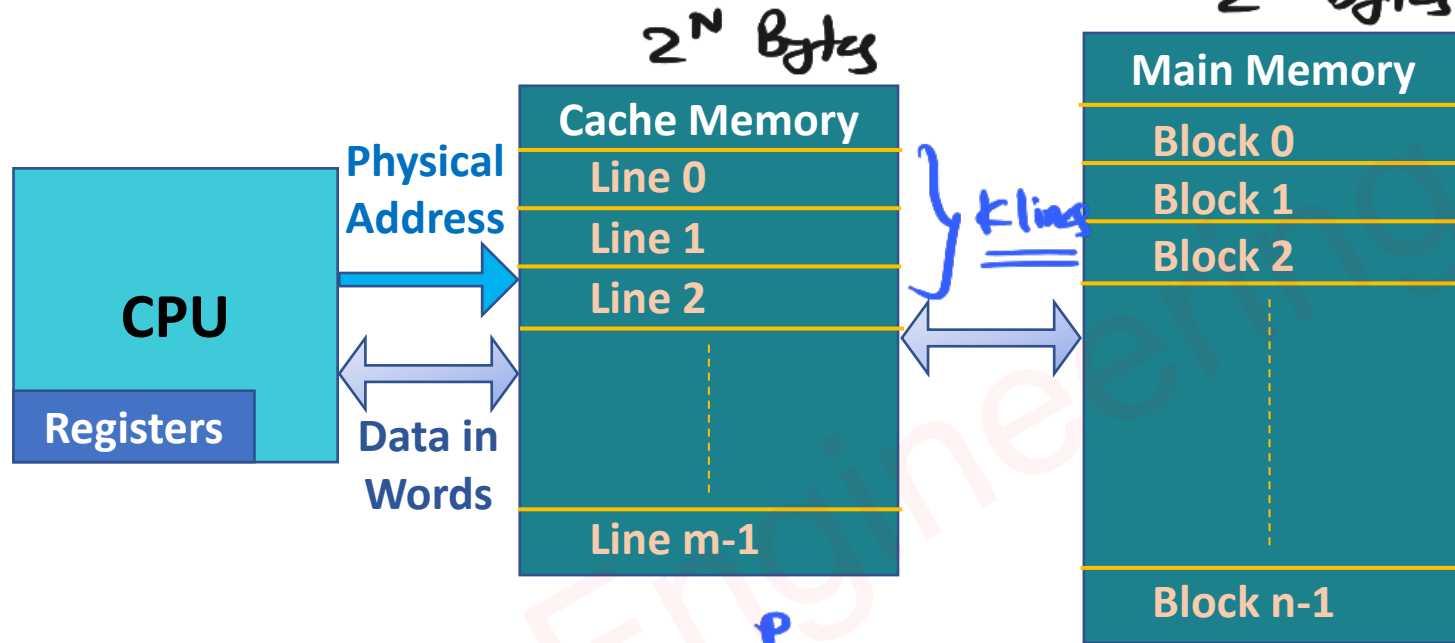
❖ The size of the physical address space of a processor is 2^P bytes. The word length is 2^W bytes. The capacity of cache memory is 2^N bytes. The size of each cache block is 2^M words. For a K way set associative cache memory, the length (in number of bits) of the tag field is

- A. $P - N - \log_2 K$
- ✓ B. $P - N + \log_2 K$
- C. $P - N - M - W - \log_2 K$
- D. $P - N - M - W + \log_2 K$

$$\Rightarrow \text{Size of Block/line} = 2^m 2^w = 2^{m+w} \text{ Bytes}$$

$$\Rightarrow \text{lines \#} = \frac{2^N}{2^{m+w}} = 2^{N-m-w}$$

$$\Rightarrow S = \frac{2^{N-m-w}}{K}$$



$$\Rightarrow x = P - \left[\log_2 \left(\frac{2^{N-m-w}}{k} \right) + m + w \right]$$

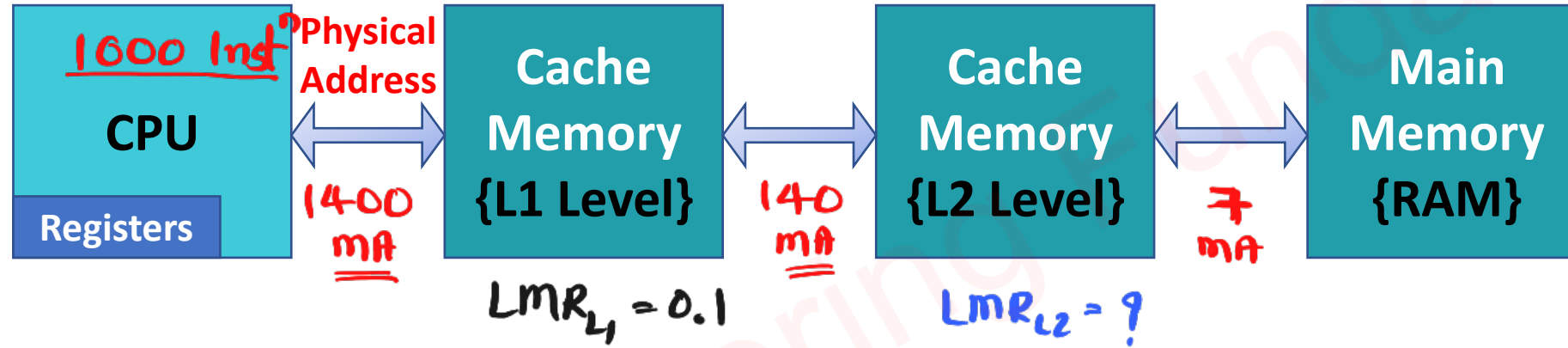
$$= P - \left[(N - \cancel{m} - \cancel{w}) - \log_2 k + \cancel{m} + \cancel{w} \right]$$

$$\boxed{x = P - N + \log_2 k}$$

Engineering Funda

Examples on Multi Level Cache Memory

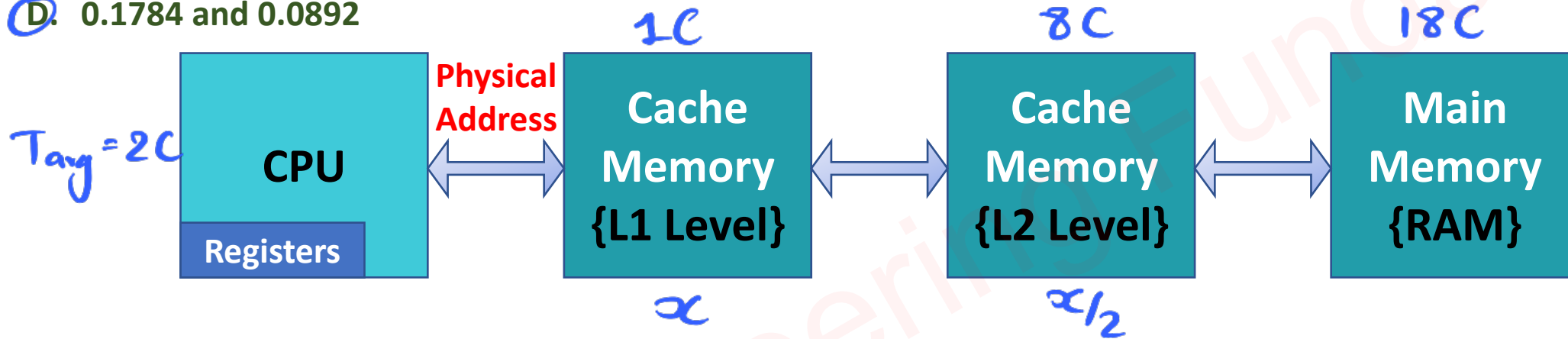
- ❖ Consider a two level cache hierarchy L1 and L2 caches. An application incurs 1.4 memory accesses per instruction on average. For this application, the miss of L1 cache is 0.1, the L2 cache experience on average 7 misses per 1000 instructions generated by CPU. The miss rate of L2 expressed correct to two decimal places is0.05....



$$\Rightarrow LMR_{L2} = \frac{7}{140} = \frac{1}{20} = 0.05$$

❖ In a two level cache system, the access times of L1 and L2 are 1 and 8 clock cycles, respectively. The miss penalty from the L2 cache to main memory is 18 clock cycles. The miss rate of L1 cache is twice that of L2. The average memory access time of the this cache system is 2 cycles. The miss rate of L1 and L2, respectively are :

- Ⓐ 0.111 and 0.056 ✓
- ✗ Ⓑ. 0.056 and 0.111
- ✗ Ⓒ. 0.0892 and 0.1784
- Ⓓ 0.1784 and 0.0892



$$\rightarrow T_{avg} = 2C = 1 + \alpha(8) + \alpha\left(\frac{\alpha}{2}\right)18$$

$$\Rightarrow 2 = 1 + 8\alpha + 9\alpha^2$$

$$\Rightarrow 9\alpha^2 + 8\alpha - 1 = 0$$

$$\Rightarrow 9\alpha^2 + 9\alpha - \alpha - 1 = 0$$

$$\Rightarrow 9\alpha(\alpha+1) - 1(\alpha+1) = 0$$

$$\Rightarrow \alpha = 1/9 \quad \text{or} \quad \alpha = -1 \quad \times$$

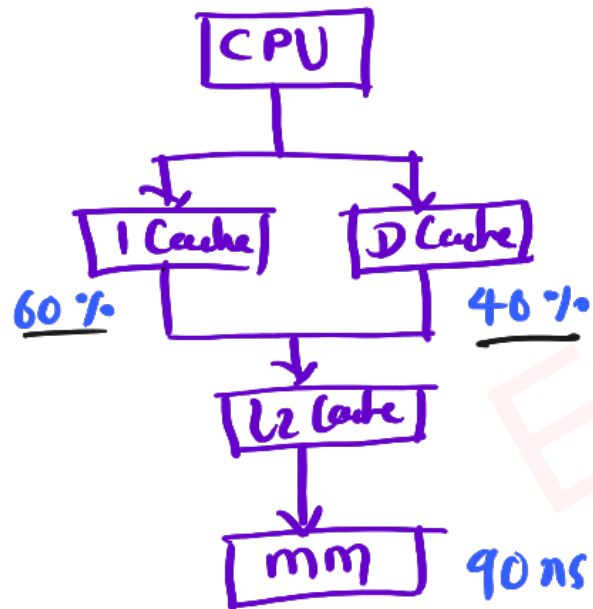
✓

- ❖ The read access times and the hit ratios for different caches in a memory hierarchy are as given below:
- ❖ The read access time of main memory is 90 ns. Assume that the caches use the referred word first read policy and the writeback policy. Assume that all the caches are direct mapped caches. Assume that the dirty bit is always 0 for all the blocks in the caches. In execution of a program, 60% of memory reads are for instruction fetch and 40% are for memory operand fetch. The average read access time in nanoseconds is

Cache	Read Access time in ns	Hit ratio
I cache	2	0.8
D cache	2	0.9
L2 cache	8	0.9

$$\Rightarrow T_{avg} = 5.4 \times 0.6 + 3.7 \times 0.4$$

$$= 4.72 \text{ nsec}$$



→ For Inst, "

$$T_{avg_i} = 2 + 0.2 \times 8 + 0.2 \times 0.1 \times 90$$

$$= 2 + 1.6 + 1.8$$

$$= 5.4 \text{ nsec}$$

→ For data

$$T_{avg_d} = 2 + 0.1 \times 8 + 0.1 \times 0.1 \times 90$$

$$= 2 + 0.8 + 0.9$$

$$= 3.7 \text{ nsec}$$

Examples on Conflict Misses in Cache Memory

- ❖ Consider a 2 way set associative cache with 256 blocks and uses LRU replacement, initially the cache is empty. Conflict misses are those misses which occur due to the contention of multiple blocks for the same cache set. Compulsory misses occur due to first time access to the block. The following sequence of accesses to memory blocks is repeated 10 times. The number of conflict misses experienced by the cache is~~76~~.....

{0, 128, 256, 128, 0, 128, 256, 128, 1, 129, 257, 129, 1, 129, 257, 129}

Cache Mem.	
L0	- 256
L1	- 128
L2	- 257
L3	- 129
....	
....	

Set 0

Set 1

Cache Mem.	
L0	- 256
L1	- 128
L2	- 257
L3	- 129
....	
....	

Set 0

Set 1

$$\Rightarrow m = 256$$

$$S = \frac{m}{k} = \frac{256}{2} = 128$$

$$\Rightarrow \text{Misses} = 4 + 9 \times 8 = \underline{76}$$

Iteration - 1

0, 256, 1, 257

(4)

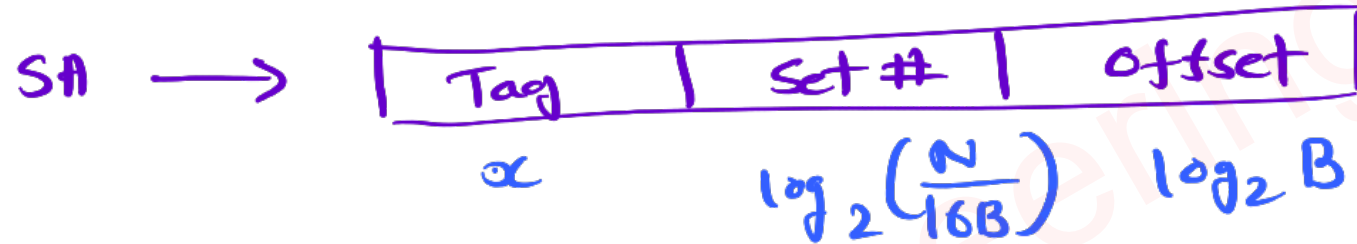
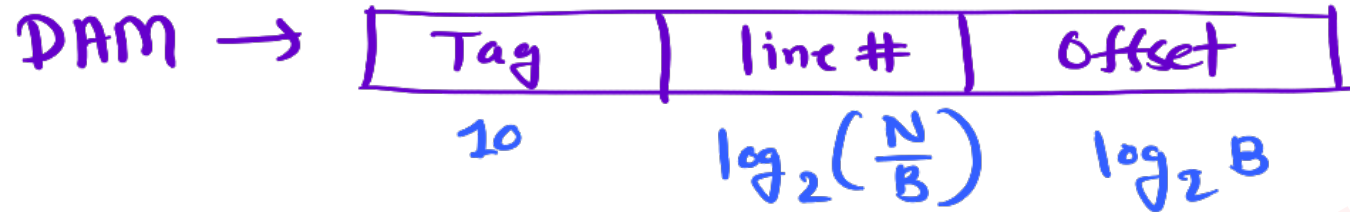
Iteration - 2

0, 0, 256, 258, 1, 1, 257, 257

(8)

Examples on mapping of Cache Memory

- ❖ A cache memory unit with a capacity of N words and block size of B words is to be designed. If it is designed as a direct mapped cache, the length of the TAG field is 10 bits. If the cache unit is now designed as a 16 way set associative cache, the length of the TAG field is¹⁴..... Bits.



$$\Rightarrow 10 + \log_2\left(\frac{N}{B}\right) + \cancel{\log_2 B} = x + \log_2\left(\frac{N}{16B}\right) + \cancel{\log_2 B}$$

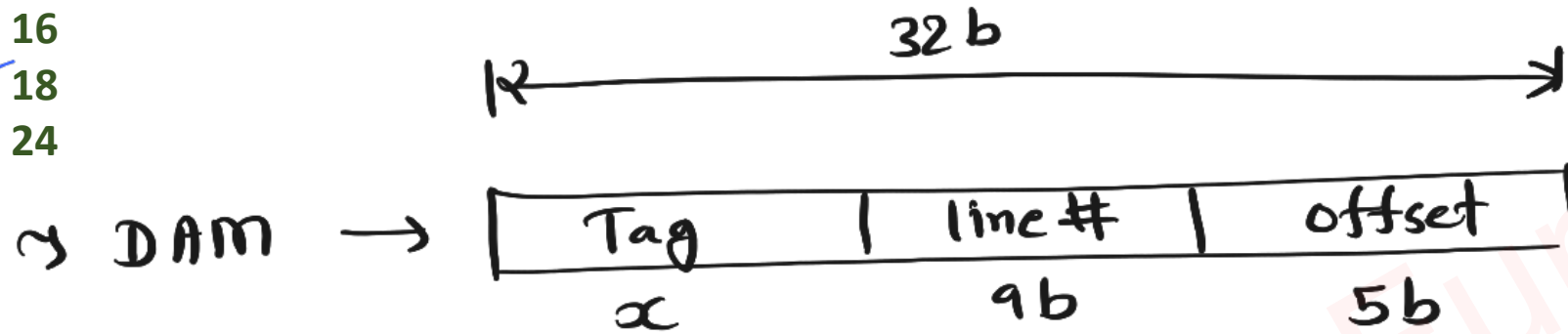
$$\Rightarrow 10 + \log_2\left(\frac{N}{B}\right) = x + \log_2\left(\frac{N}{B}\right) + \log_2\left(\frac{1}{16}\right)$$

$$\Rightarrow 10 = x - 4$$

$$\Rightarrow x = 14$$

❖ Consider a machine with byte addressable memory of 2^{32} bytes divided into blocks of size 32 bytes. Assume a direct mapped cache having 512 cache lines are used with this machine. The size of tag field in bits is18b....

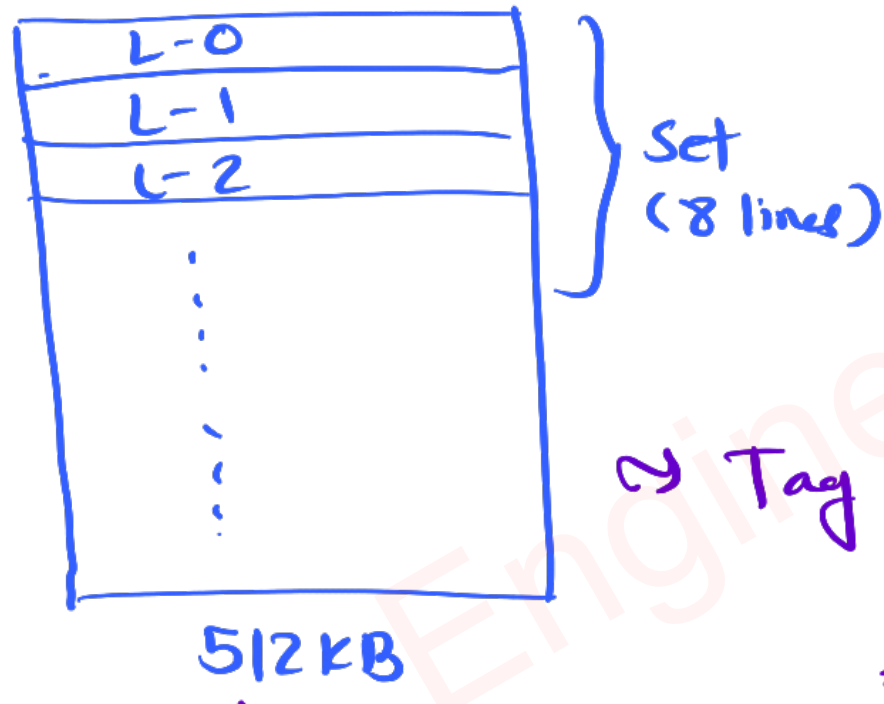
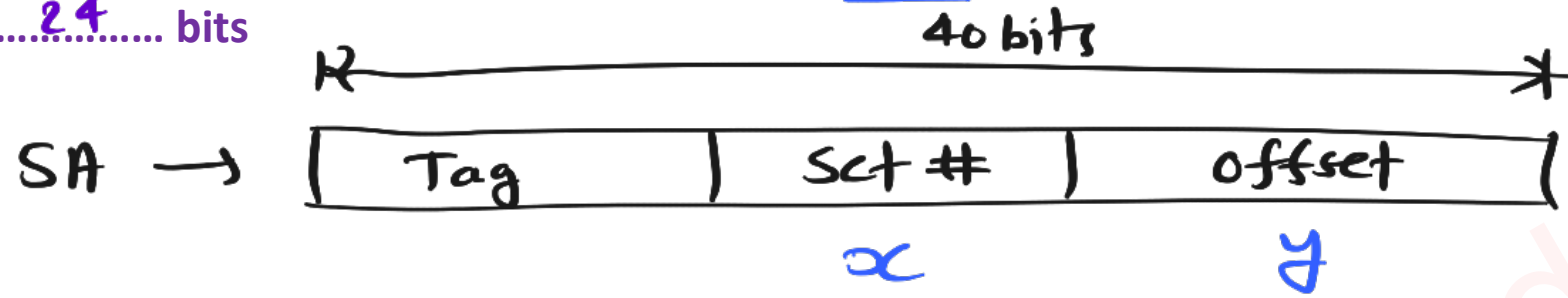
- A. 12
- B. 16
- ✓ C. 18
- D. 24



$$\rightarrow \text{Tag} = 32 - 9 - 5 \\ = 18\text{ b}$$

❖ The width of the physical address on a machine is 40 bits. The width of the tag field in a 512 KB 8 way set associative cache is 24 bits

- ✓ A. 24
- B. 20
- C. 30
- D. 40



$$\Rightarrow 2^{19} = 2^3 \times 2^x \times 2^y$$

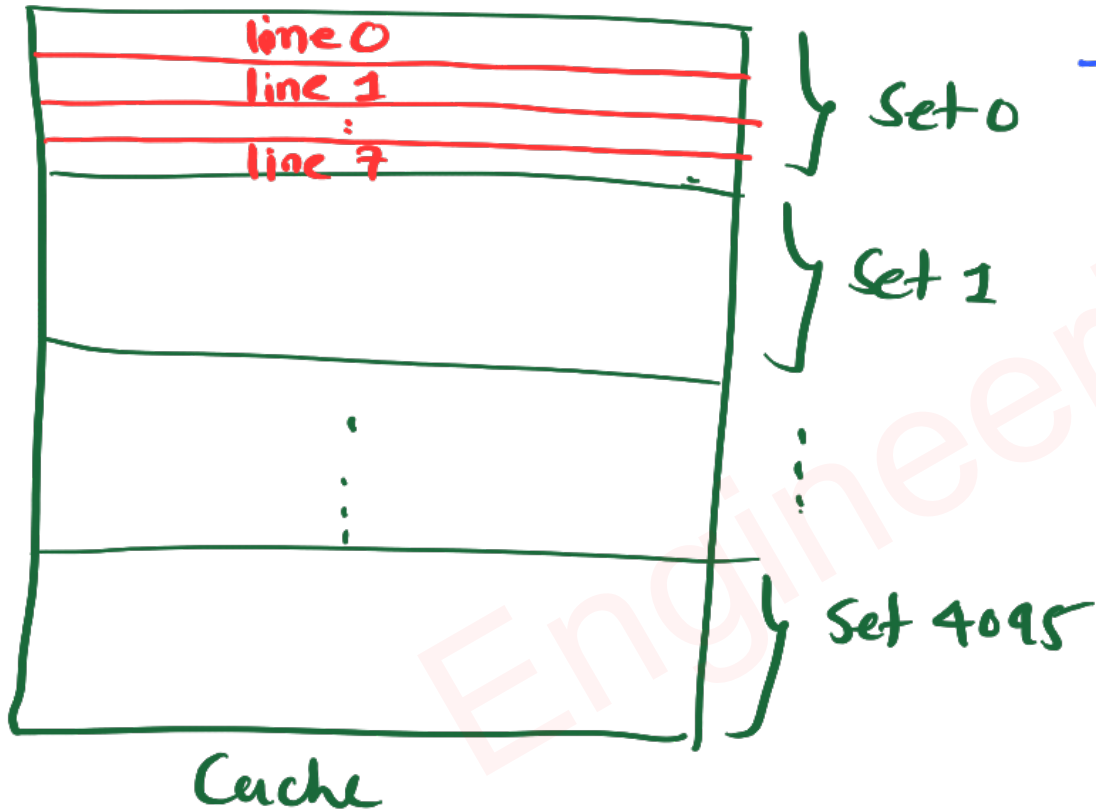
$$\Rightarrow 19 = 3 + x + y$$

$$\Rightarrow \boxed{x + y = 16}$$

$$\begin{aligned}\Rightarrow \text{Tag} &= 40 - (x + y) \\ &= 40 - 16 \\ &= 24 \text{ b.}\end{aligned}$$

Examples on Cache Memory

- ❖ If an 8 way set associative cache is made up of 32 bits words, 4 words per line and 4096 sets. How big is the cache in KB?

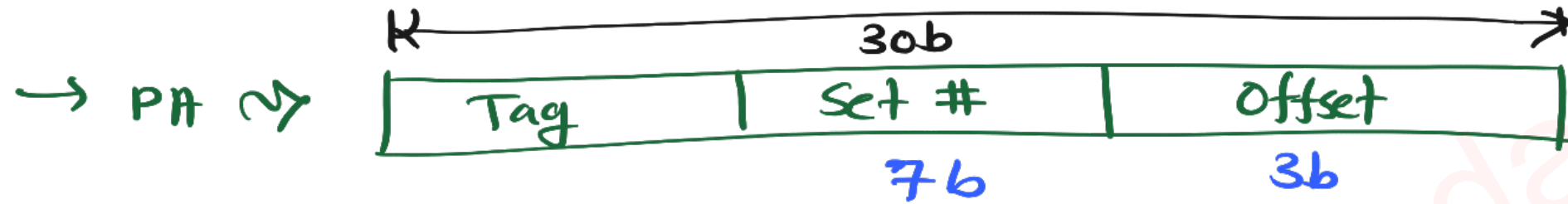


$$\rightarrow \text{Size of line} = 4w = 4 \times 4B = 16B$$

$$\rightarrow \text{Size of Set} = 16B \times 8 = 128B$$

$$\begin{aligned} \rightarrow \text{Size of Cache} &= 128 \times 4096B \\ &= 4 \times 128KB \\ &= 512KB \end{aligned}$$

- ❖ A 4 way set associative cache memory unit with a capacity of 16KB is built using a block size of 8 words. The word length is 32 bits. The size of the physical address space is 4GB. The number of bits for the TAG field is20b....



$$\begin{aligned} \approx \text{mm} &= 4 \text{ KB} \\ &= 1 \text{ word} \end{aligned}$$

$$\rightarrow \text{Size of block} = 8 \text{ w} = 2^3 \text{ w}$$

$$\rightarrow \text{line \#} = \frac{16 \text{ KB}}{8 \text{ w}} = \frac{16 \times 2^{16}}{8 \times 4} = 512$$

$$\rightarrow S = \frac{\text{line \#}}{K} = \frac{512}{4} = 128 = 2^7$$

$$\begin{aligned} \rightarrow \text{Tag} &= 30 - 7 - 3 \\ &= 20 \text{ b} \end{aligned}$$

- ❖ Consider a 2 way set associative cache memory with 4 sets and total 8 cache blocks (0-7) and a main memory with 128 blocks (0-127). What memory blocks will be present in the cache after the following sequence of memory block references if LRU policy is used for cache block replacement:

$S_0 \ S_1 \ S_3 \ S_1 \ S_3 \ S_0 \ S_0 \ S_3$
0 5 3 9 7 0 16 55
 LRU MRU

- ❖ Initially cache is empty.

- ~~A.~~ 0 3 5 7 16 55
~~B.~~ 0 3 5 7 9 16 55
~~C.~~ 0 5 7 9 16 55
~~D.~~ 3 5 7 9 16 55

L-0 = 0	Set 0
L-1 = 16	
L-2 = 5	Set 1
L-3 = 9	
L-4	Set 2
L-5	
L-6 = 55	Set 3
L-7 = 7	

Examples on Cache Memory

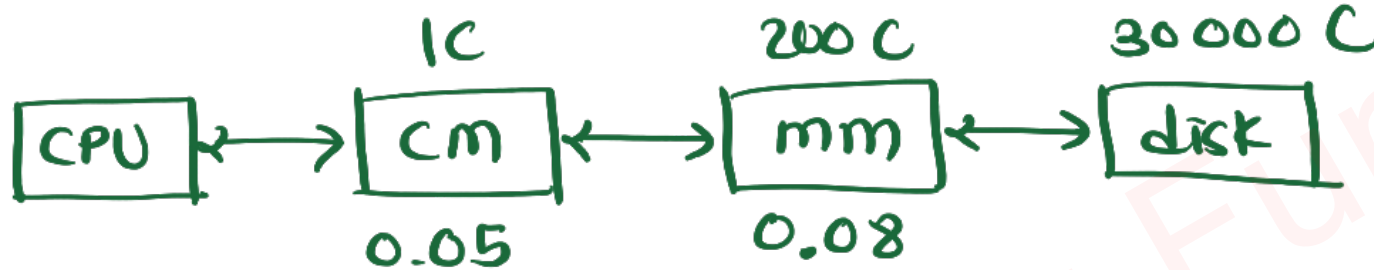
❖ The cache takes 1 cycle to access and has 5% miss rate, main memory takes 200 cycles to access and has an 8% miss rate and the disk takes 30000 cycles to access. The average memory access time in cycles for the given cache is

A. 128

B. 135

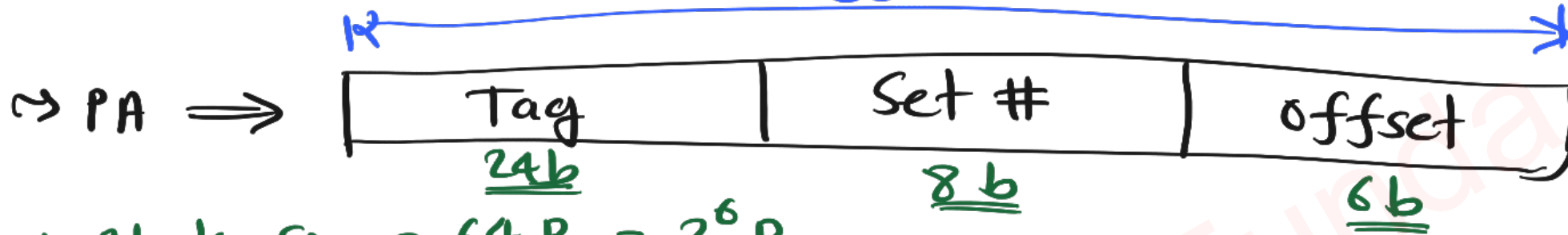
✓ C. 131

D. 140



$$\begin{aligned} \Rightarrow T_{avg} &= 1 + 0.05 \times 200 + 0.05 \times 0.08 \times 30000 \\ &= \underline{\underline{131}} \end{aligned}$$

❖ Consider a 4 way set associative cache memory unit with a capacity 64 KB with each block size 64B. The size of the physical address space is 256GB. The physical address 0E1B01AA050 (in Hex) is mapping to the set cache is (in decimal) ...129.....



⇒ Block Size = 64 B = 2^6 B

⇒ line # = $\frac{64 \text{ KB}}{64 \text{ B}} = 1024$

⇒ Set # = $\frac{\text{line \#}}{k} = \frac{1024}{4} = 256 = 2^8$

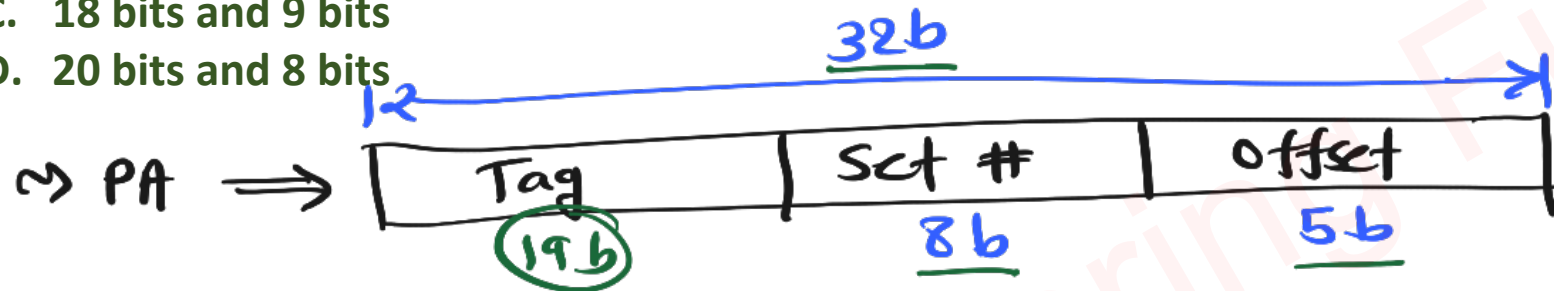
..... 1010 1010 0000 0101 0000

tag	Set #	offset
	= 129	= 16

Examples on Cache Memory

❖ A certain processor uses a 8 way associative cache of size of 64KB, the cache block size is 32 bytes. Assume that the main memory is byte addressable and uses a 32 bit address. How many bits are required for the tag and the set index fields respectively in the address generated by the processor?

- A. 19 bits and 8 bits ✓
- B. 16 bits and 11 bits
- C. 18 bits and 9 bits
- D. 20 bits and 8 bits



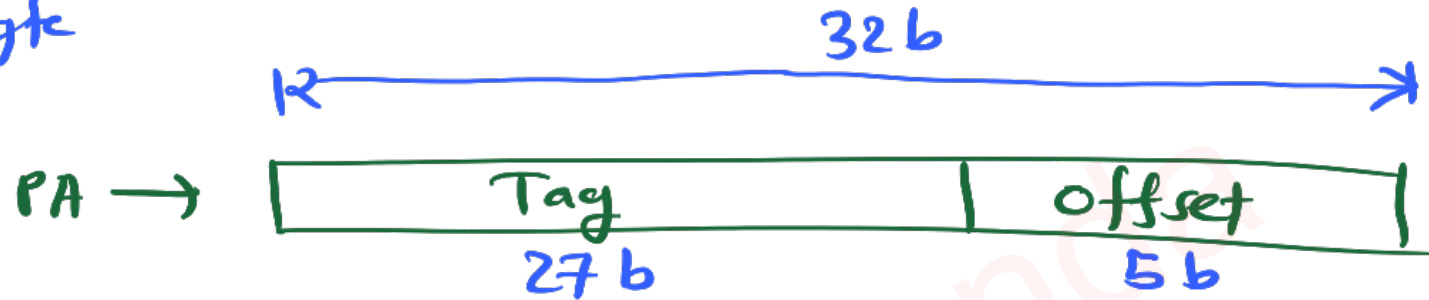
$\Rightarrow$ Block Size = 32 Bytes
 $= 2^5 \text{ B}$

$\Rightarrow$ line # = $\frac{64 \text{ KB}}{32} = 2048$

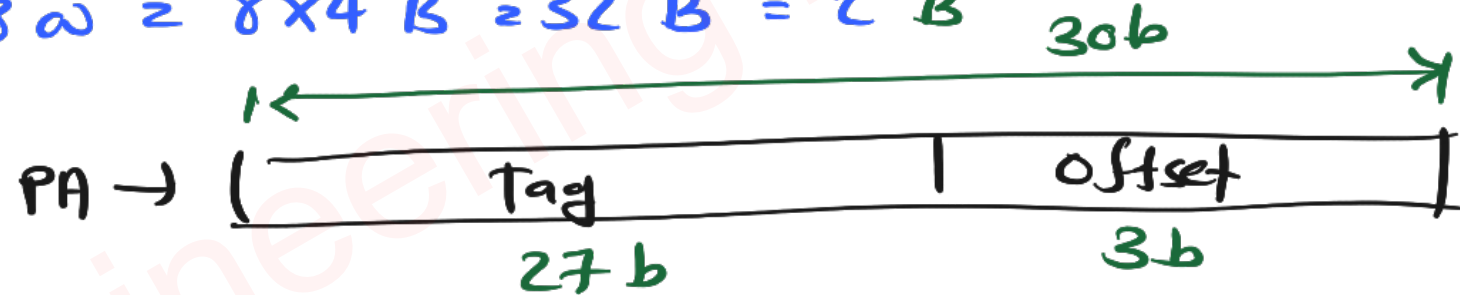
$\Rightarrow$ Set # = $\frac{\text{line \#}}{k} = \frac{2048}{8} = 256 = 2^8$

- ❖ Consider a fully associative cache memory unit with a capacity 16 KB is built using a block size of 8 words. The word length is 32 bits. The size of the physical address space is 4GB. The memory occupied by the TAG bits in cache (in bytes) is 1728 Byte

Tag	data
	L-0
	L-1



⇒ Size of block = 8 w = $8 \times 4 \text{ B} = 32 \text{ B} = 2^5 \text{ B}$



⇒ mm = 4 GB
= 1 Gw

⇒ Size of block = 8 w = 2^3 w

⇒ line # = $\frac{16 \text{ KB}}{2,32 \text{ B}} = 512$

→ Size of Tag bits

= $512 \times 27 \text{ (b)}$

= $\frac{512 \times 27 \text{ (B)}}{8} = 1728 \text{ Byte}$